Prof Xiaowei Huang

https://cgi.csc.liv.ac.uk/~xiaowei/

(Attendance Code: **482339**)

Lecture 18 -- Convolutional Neural Networks

Attendance Code: 482339

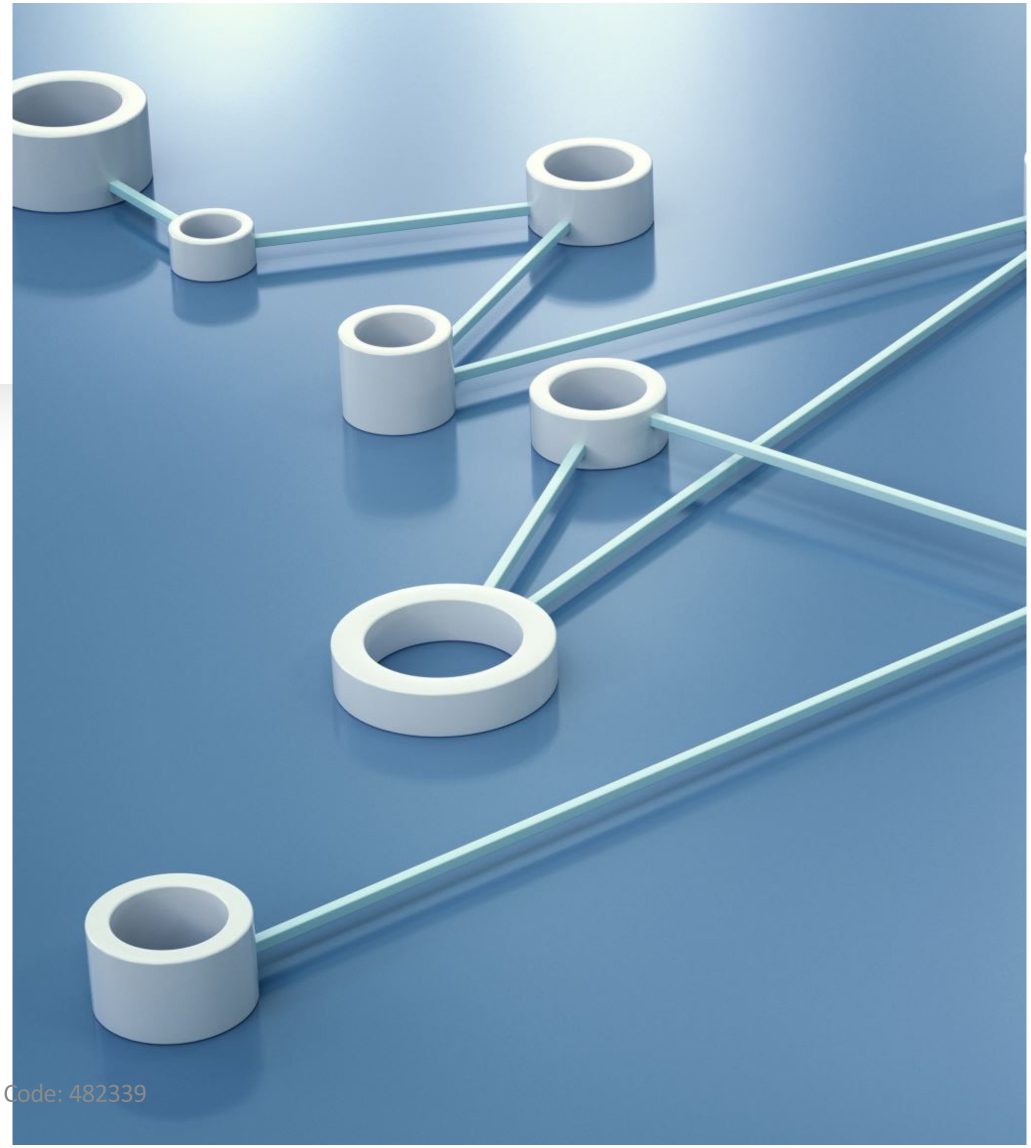
Up to now,

- Traditional Machine Learning Algorithms
- Adversarial Attack and Defence
- Deep learning
  - Introduction to Deep Learning
  - Functional view and features
  - Backward and forward computation (including backpropogation and chain rule)

Attendance Code: 482339

Topics

- Convolutional neural networks (CNN)
  - Fully-connected
  - Convolutional Layer
  - Advantage of Convolutional Layer
  - Zero-padding Layer
  - ReLU Layer
  - Pooling Layer
  - Residual Block
  - Dropout
  - BatchNorm Layer
  - Softmax Layer
- Preprocessing data
- Example
  - LeNet



Attendance Code: 482339

```
model = Sequential()

model.add(Convolution2D(nb_filters, nb_conv, nb_conv,
                        border_mode='valid',
                        input_shape=(1, img_rows, img_cols)))
model.add(Activation('relu'))
model.add(Convolution2D(nb_filters, nb_conv, nb_conv))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(nb_pool, nb_pool)))
model.add(Dropout(0.25))

model.add(Flatten())
model.add(Dense(128))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(nb_classes))
model.add(Activation('softmax'))

model.compile(loss='categorical_crossentropy',
              optimizer='adadelta',
              metrics=['accuracy'])
```

Convolutional layer

ReLU layer

Convolutional layer

ReLU layer

Maxpooling layer

Dropout layer: for regularisation

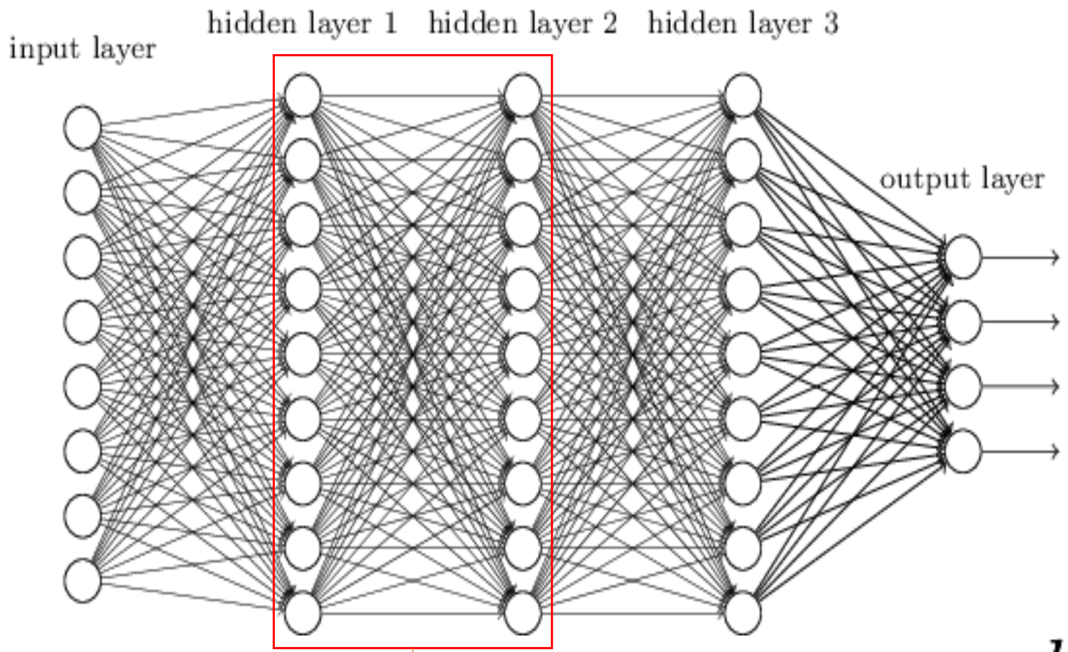
Flatten layer: from convolutional to fully-connected

Fully-connected layer

Attendance Code: 482339

Fully-connected Layer

Attendance Code: 482339



$$h = \sigma(W^T x + b)$$

All neurons of neighbouring layers are connected

Attendance Code: 482339

Convolutional Layer:

Unlike a fully connected layer, which learns separate weights for every input feature, a convolutional layer learns small, shared filters that slide across the input to detect local patterns such as edges, textures, or shapes.

We use convolutional layers instead of fully connected layers because they drastically reduce the number of parameters while preserving spatial structure, allowing the model to efficiently learn local patterns and achieve better generalization in image data.

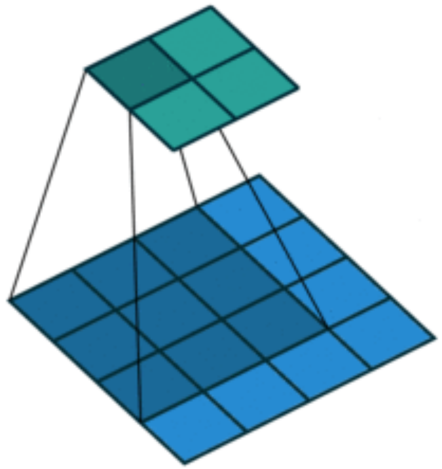
Convolutional neural networks

- Strong empirical application performance
- Convolutional networks: neural networks that use convolution in place of general matrix multiplication in at least one of their layers

$$h = \sigma(W^T x + b)$$

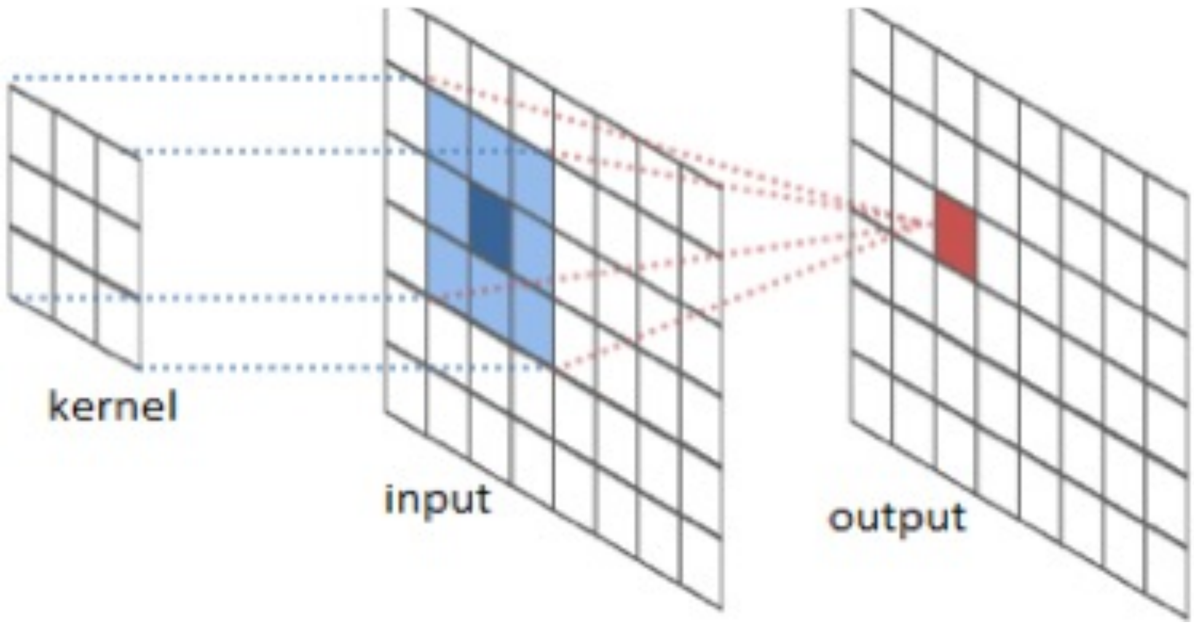
for a $\boxed{\text{specific kind of weight matrix } W}$

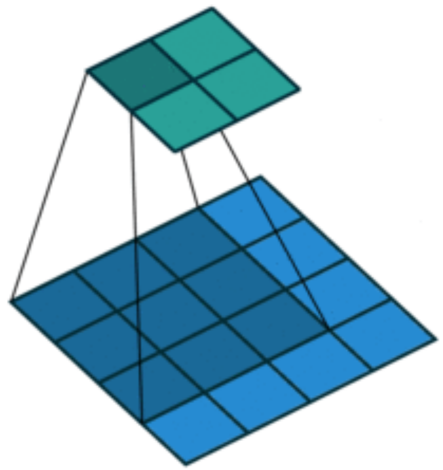
Will be explained later



Attendance Code: 482339

Convolutional layer illustrated





Attendance Code: 482339

Illustration 1

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$

x	y	z

a	b	c	d	e	f

Illustration 1

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$

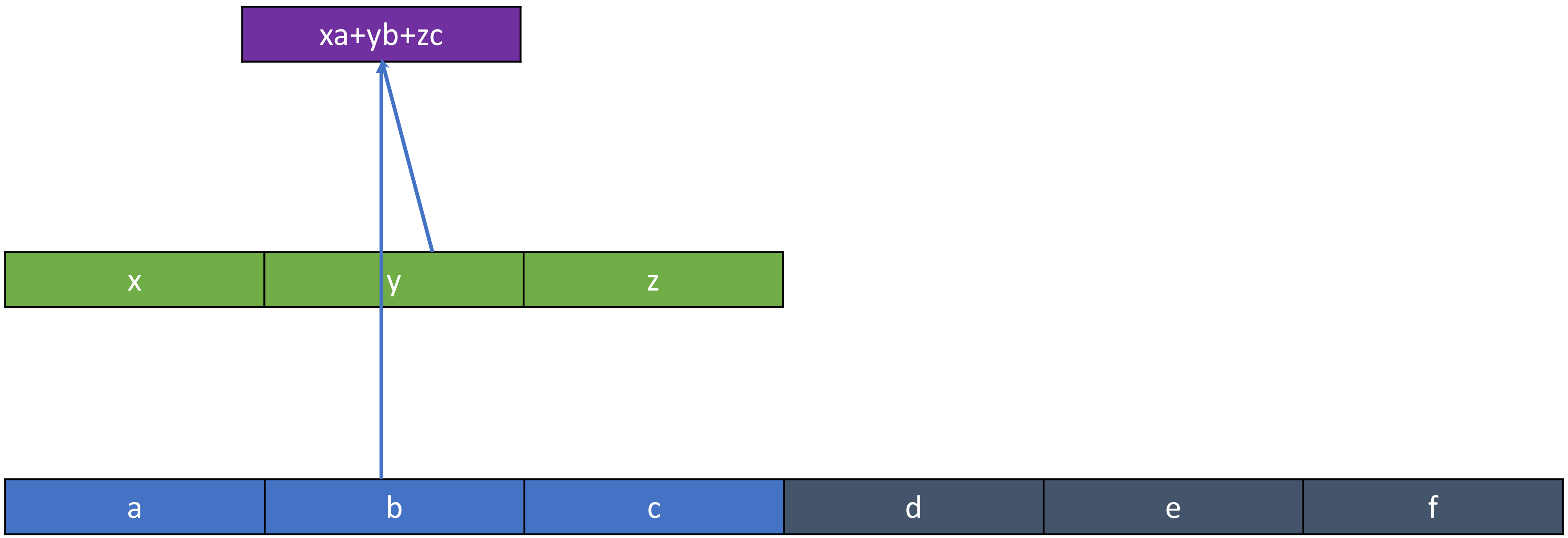


Illustration 1

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$

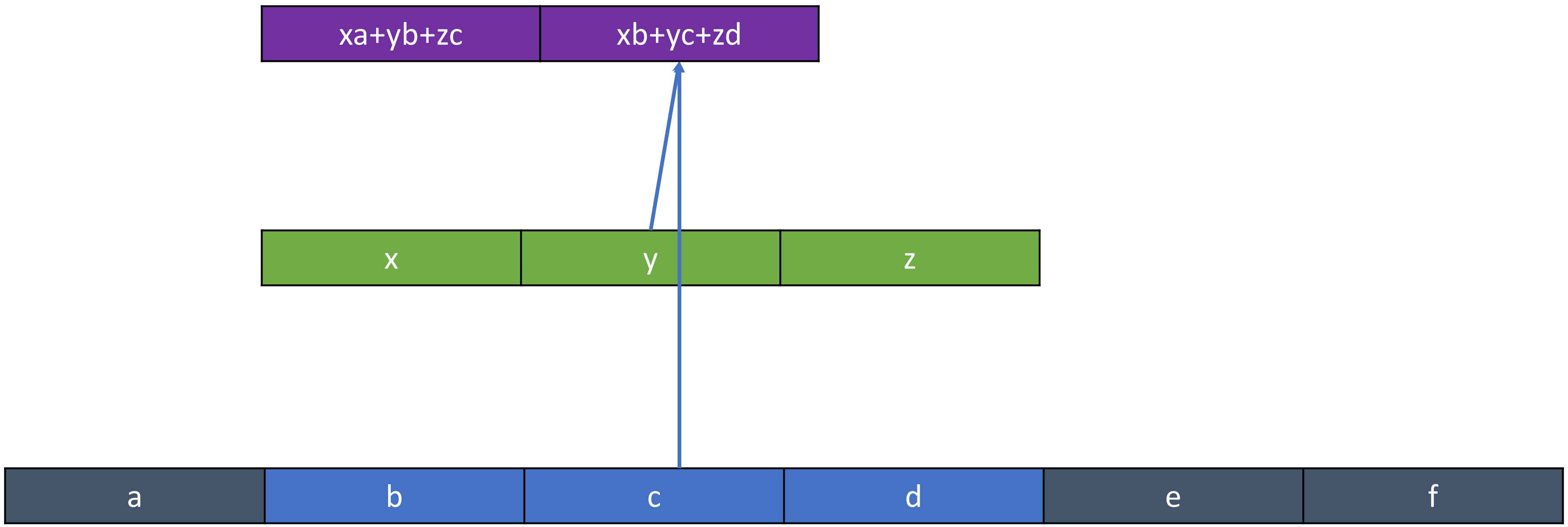
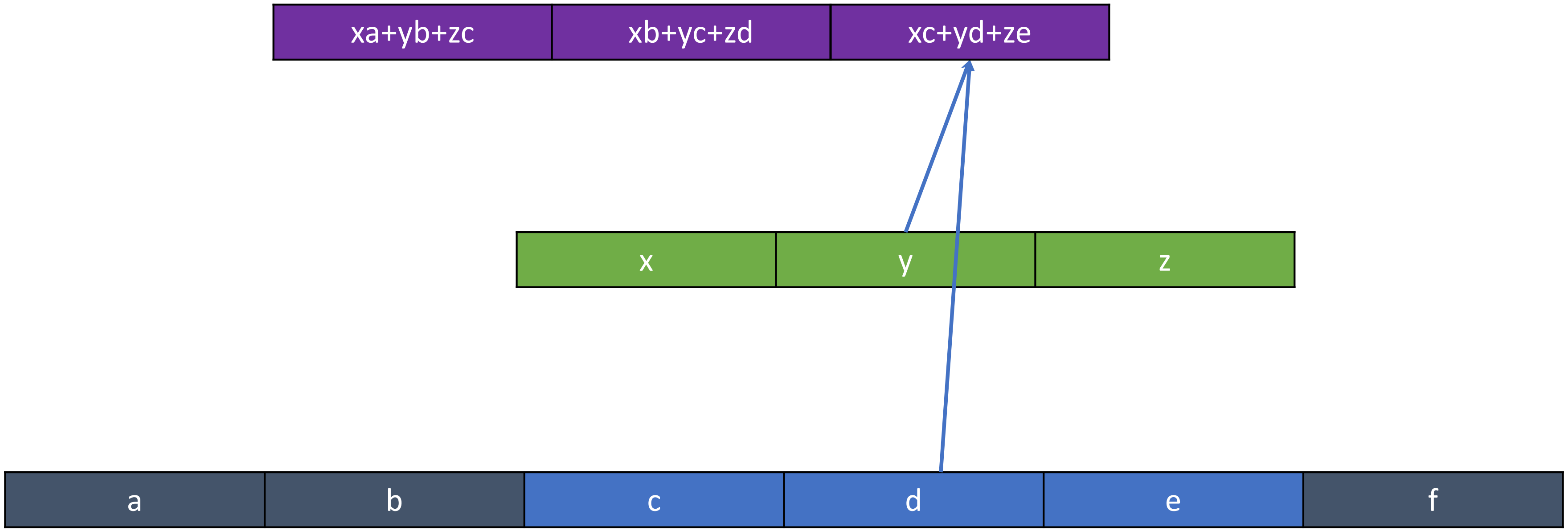


Illustration 1

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$



Attendance Code: 482339

Illustration 1

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$

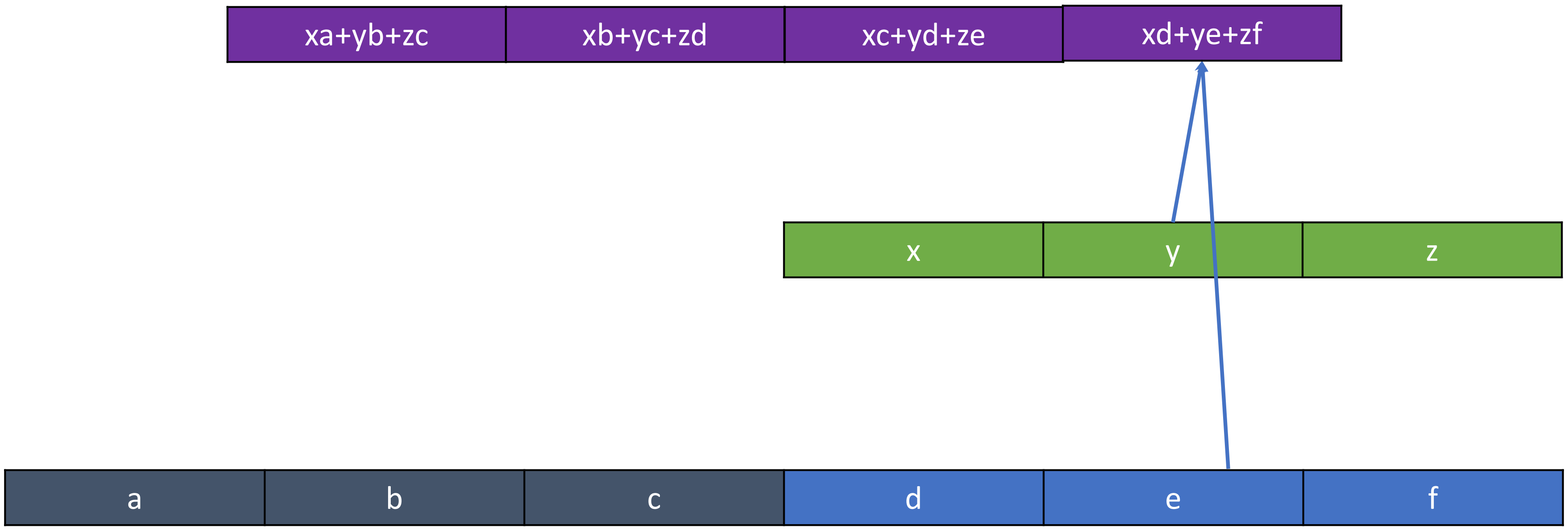
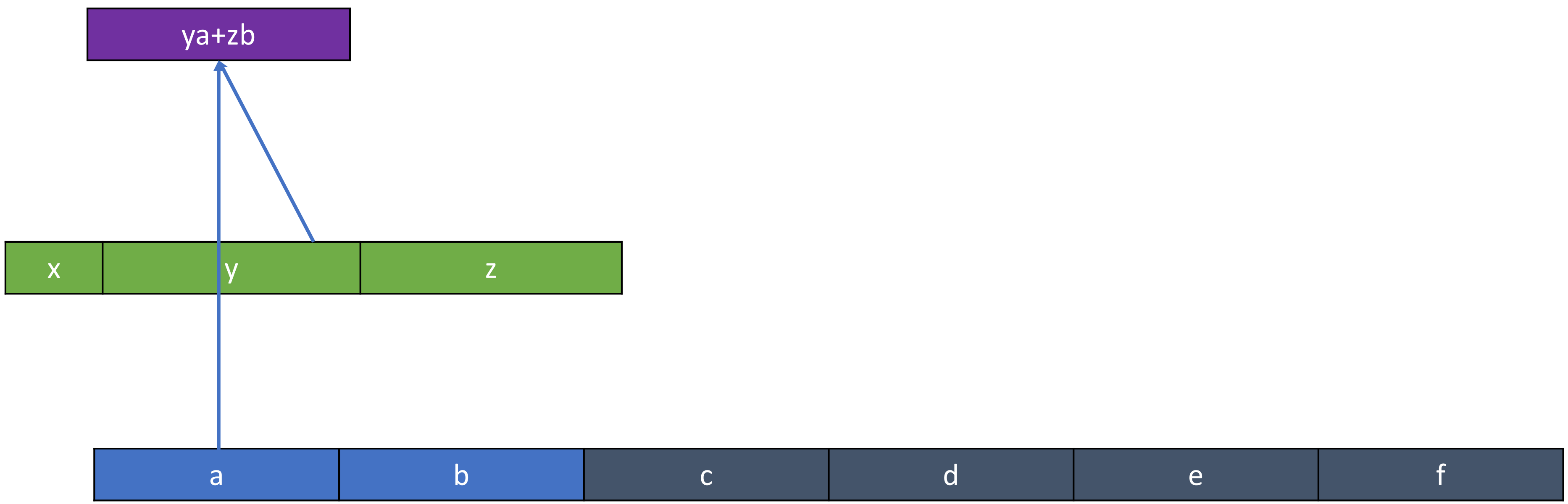


Illustration 1: boundary case

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$



Attendance Code: 482339

Illustration 1: boundary case

$$w = [z, y, x]$$
$$u = [a, b, c, d, e, f]$$

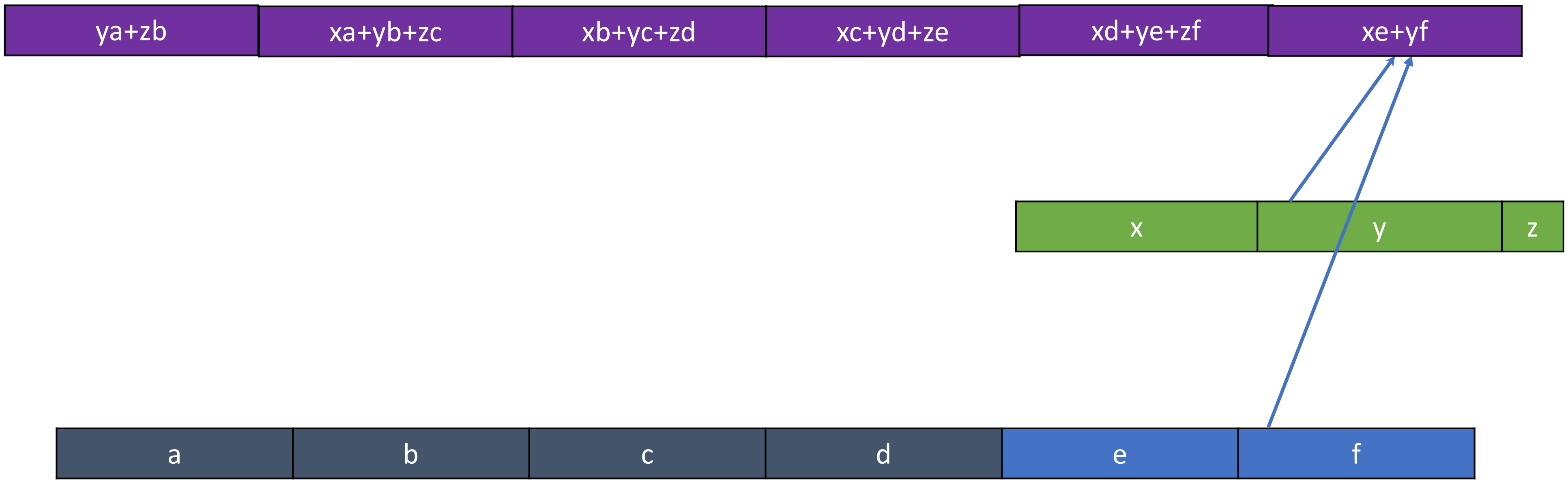
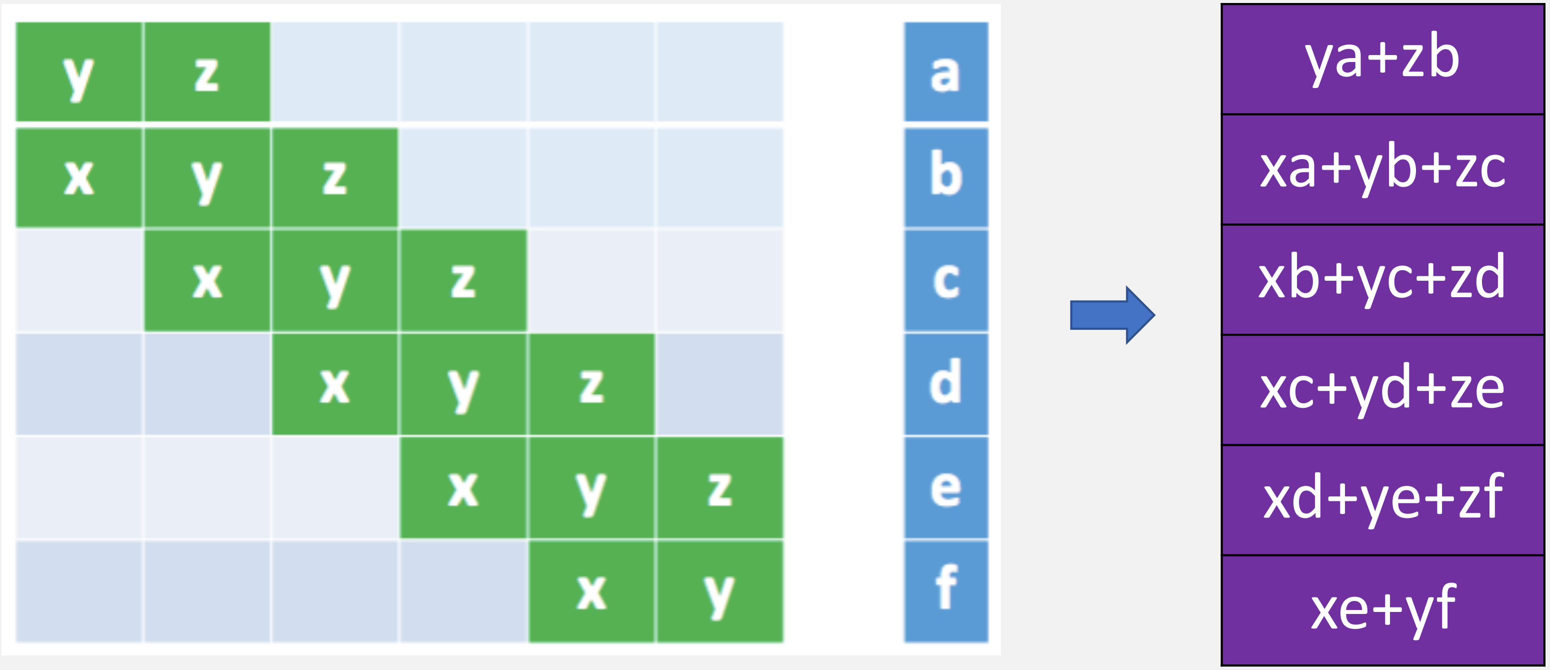
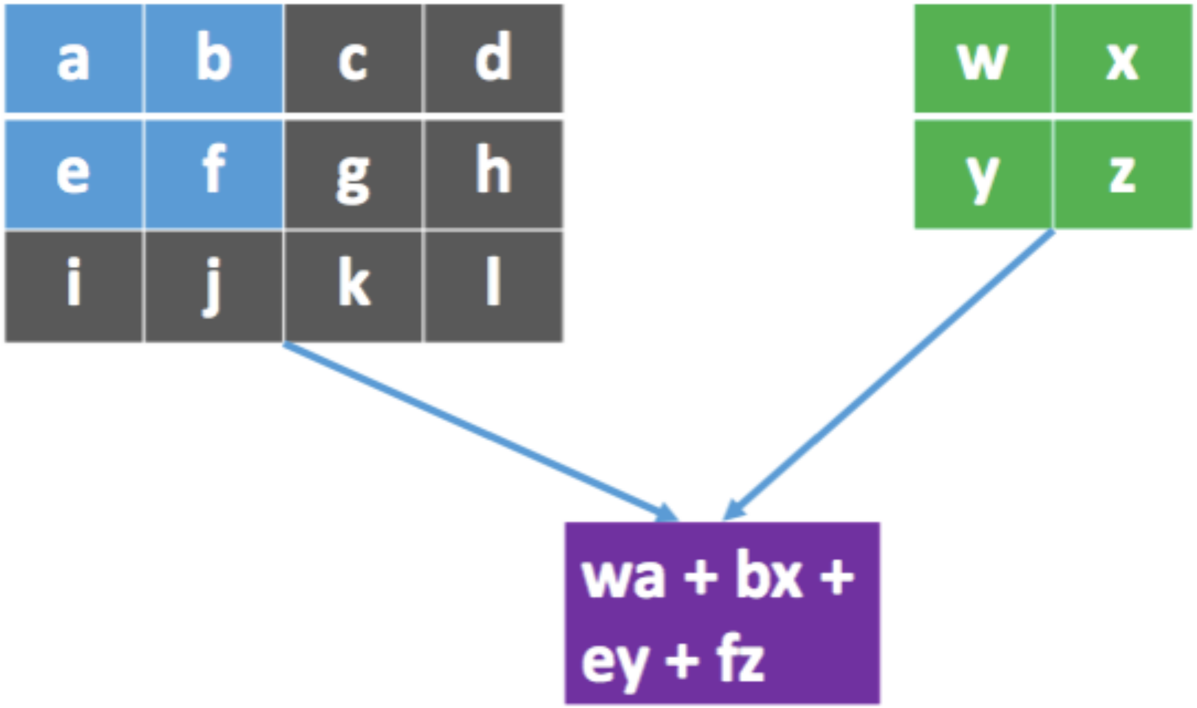


Illustration 1: as matrix multiplication



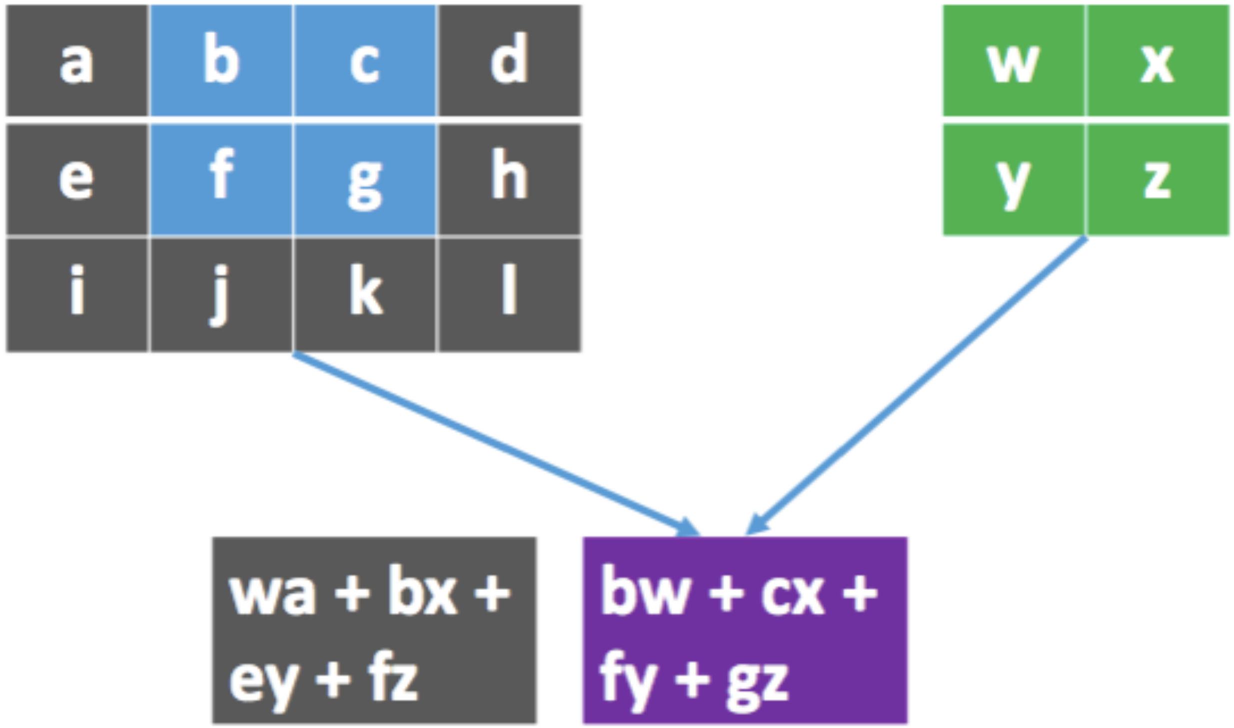
It also explains why convolutional layer is a special type of fully connected layer – weights of non-green cells are known to be 0 before training.

Illustration 2: two dimensional case



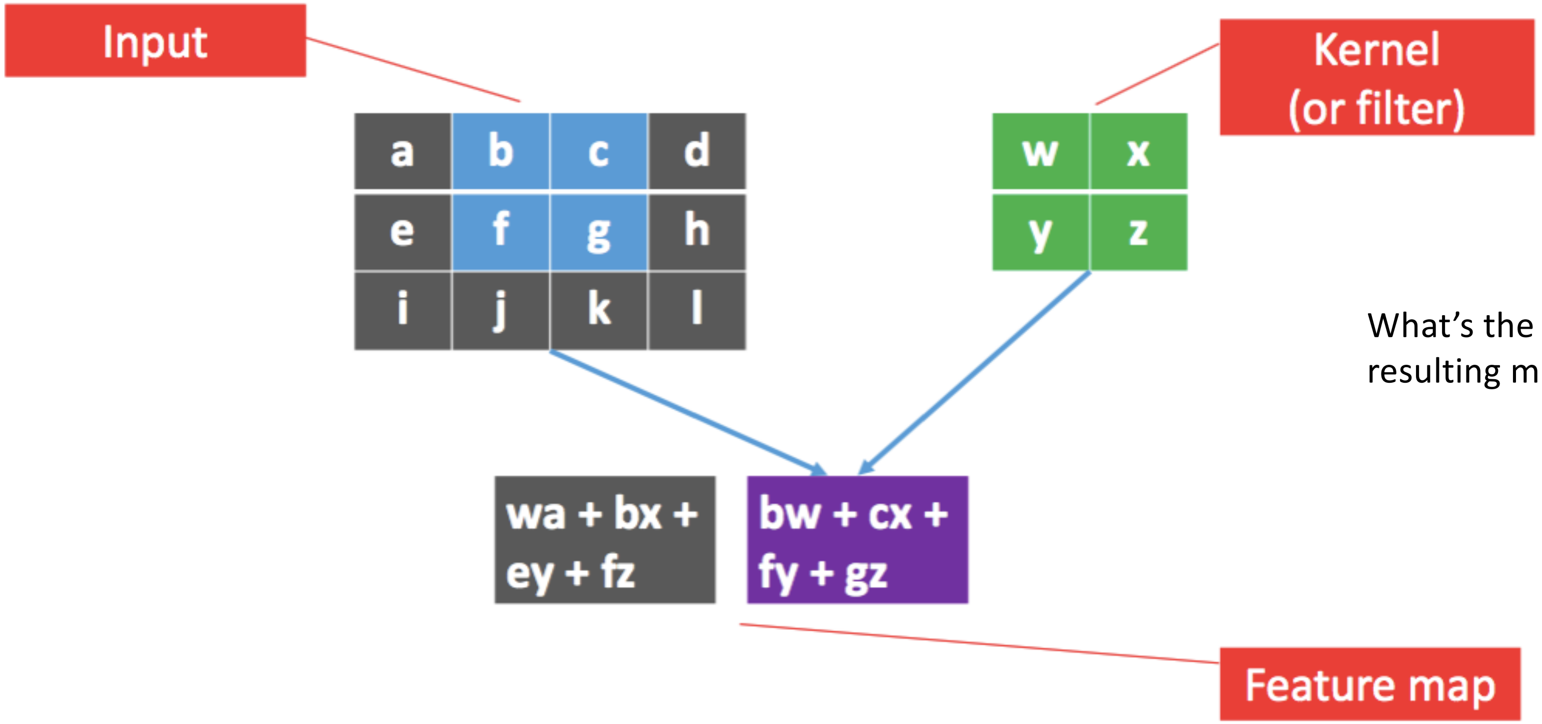
Attendance Code: 482339

Illustration 2



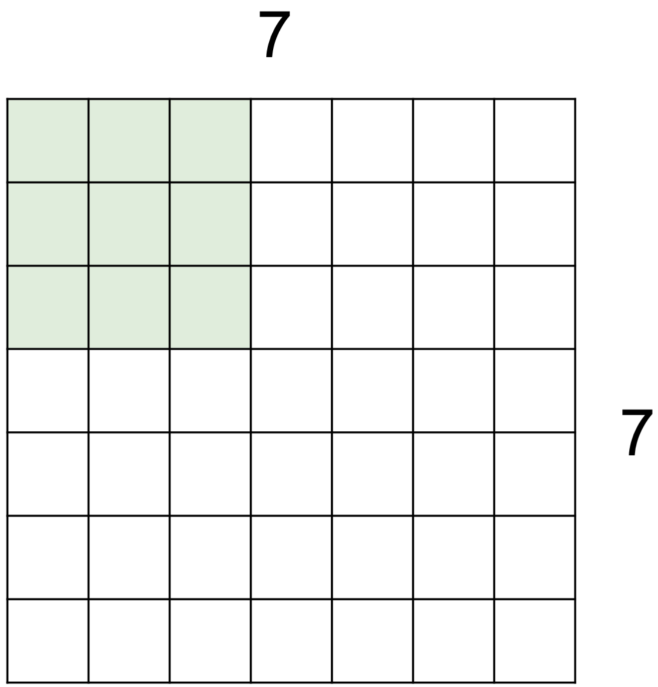
Attendance Code: 482339

Illustration 2



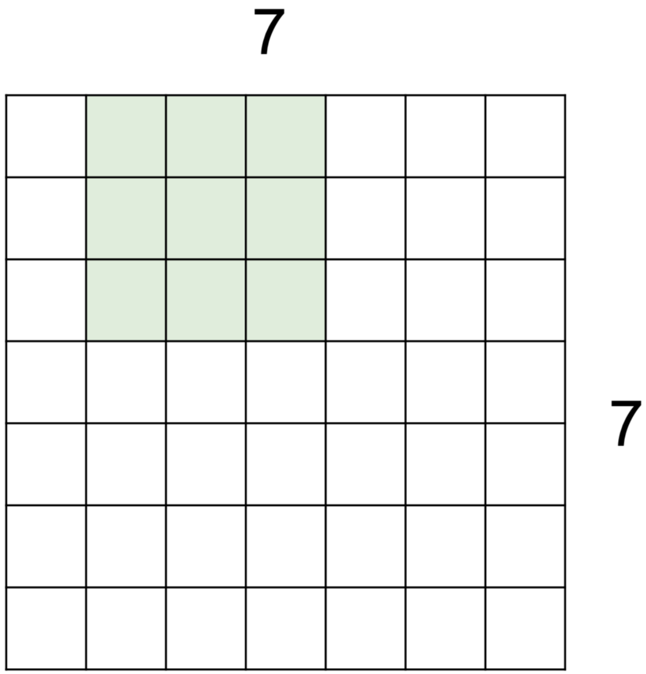
What's the shape of the resulting matrix?

A closer look at spatial dimensions:



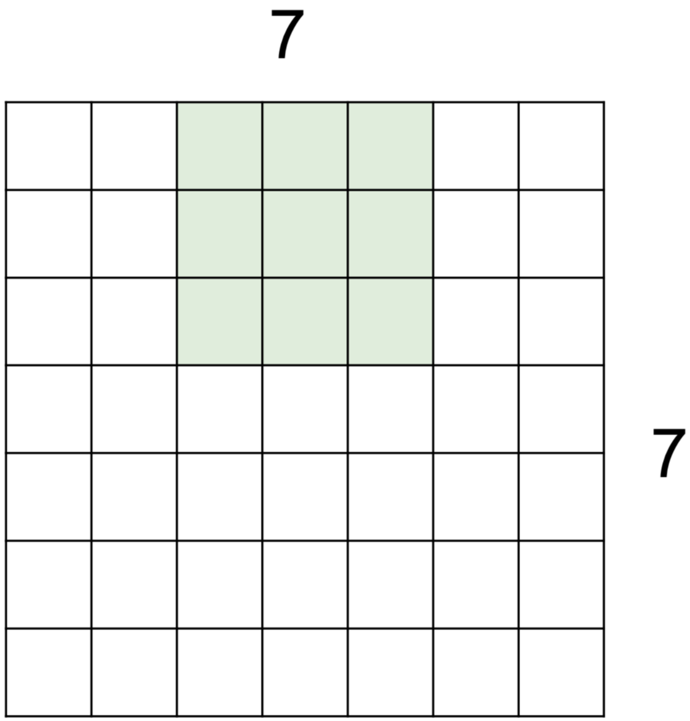
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



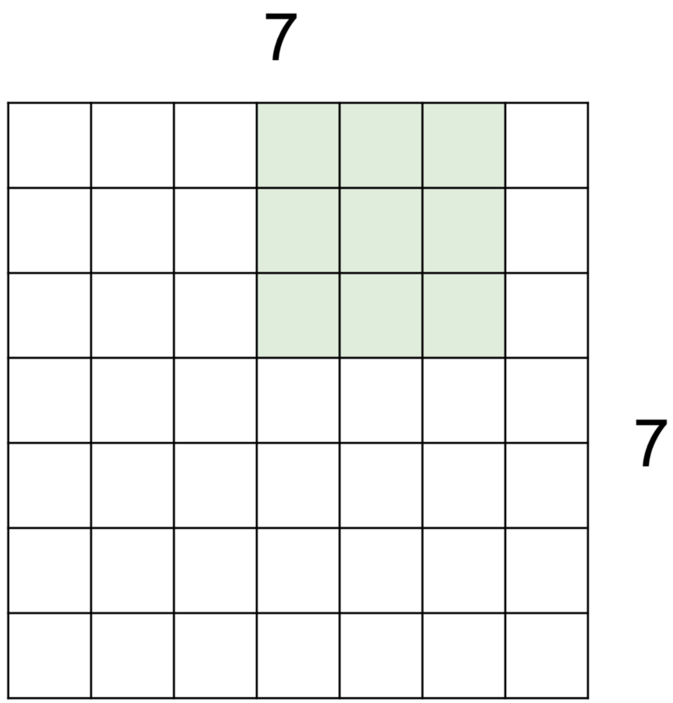
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



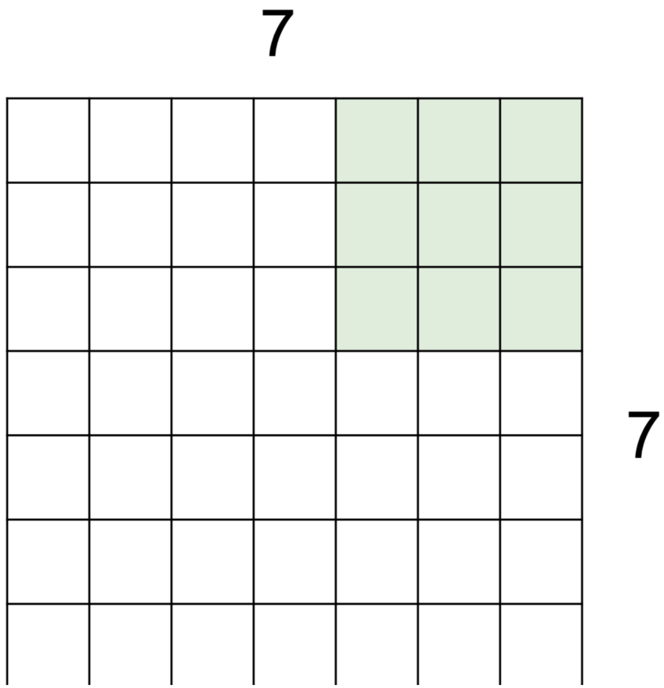
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

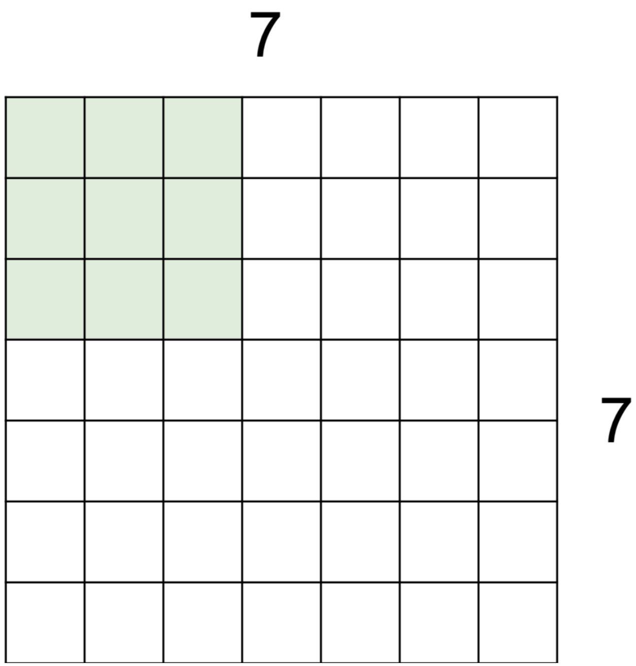
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

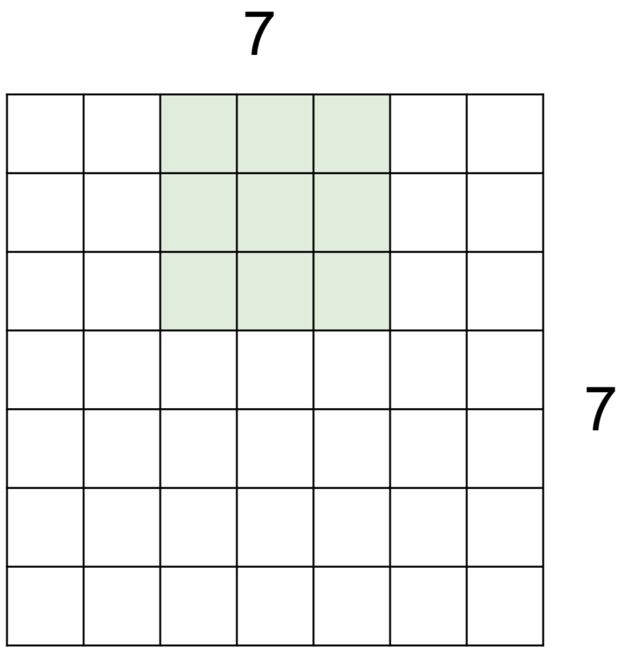
=> 5x5 output

A closer look at spatial dimensions:



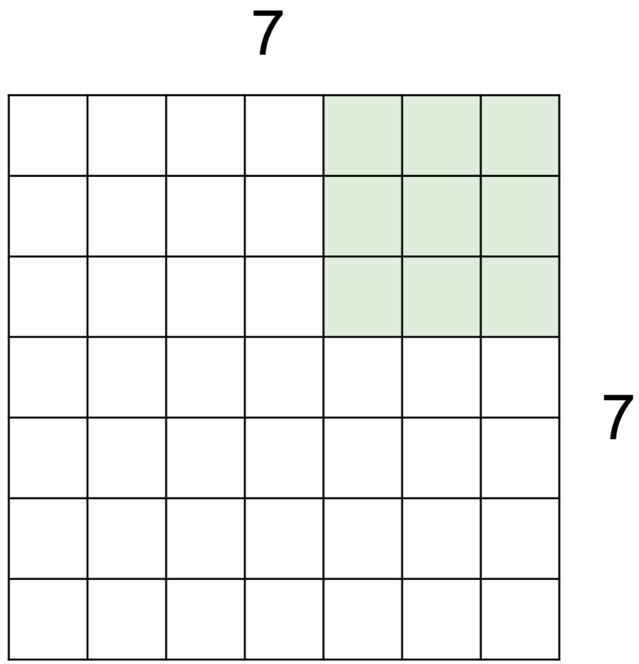
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



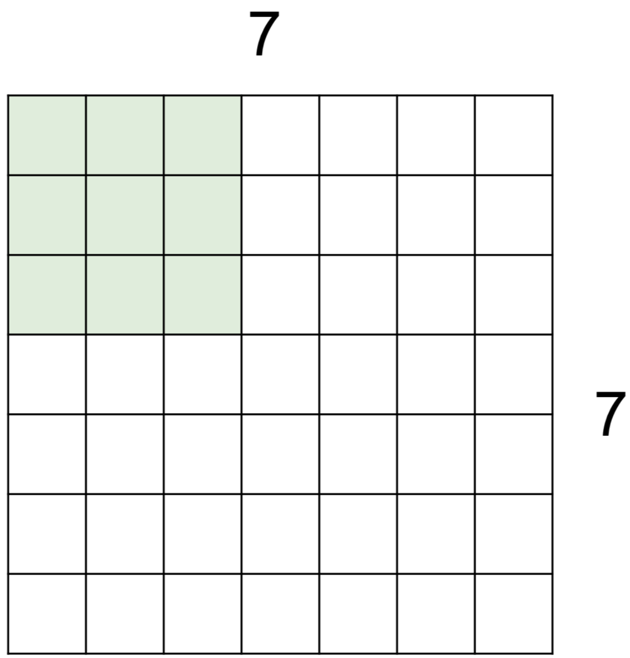
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



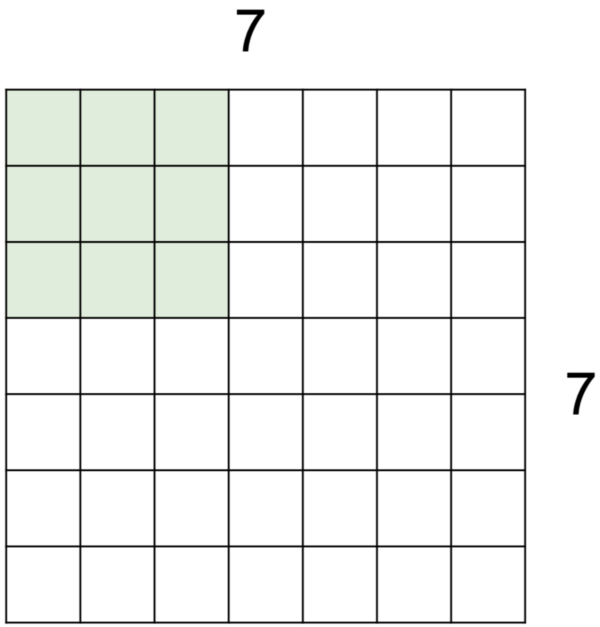
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

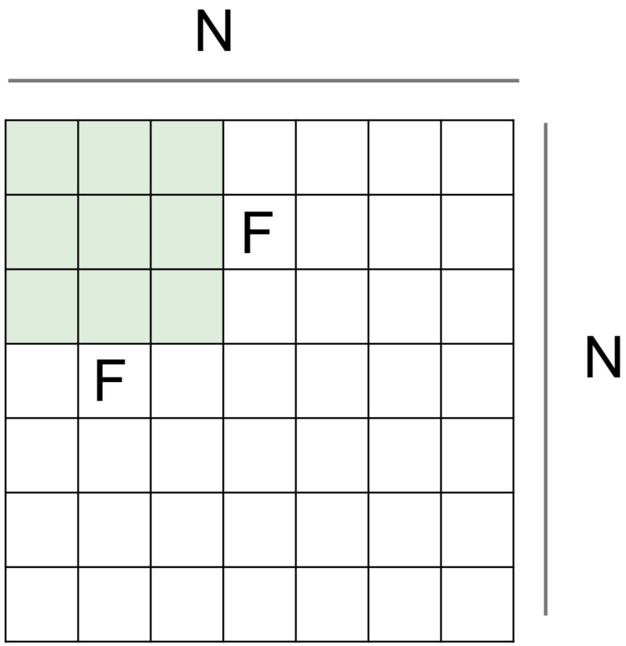
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

A closer look at spatial dimensions:



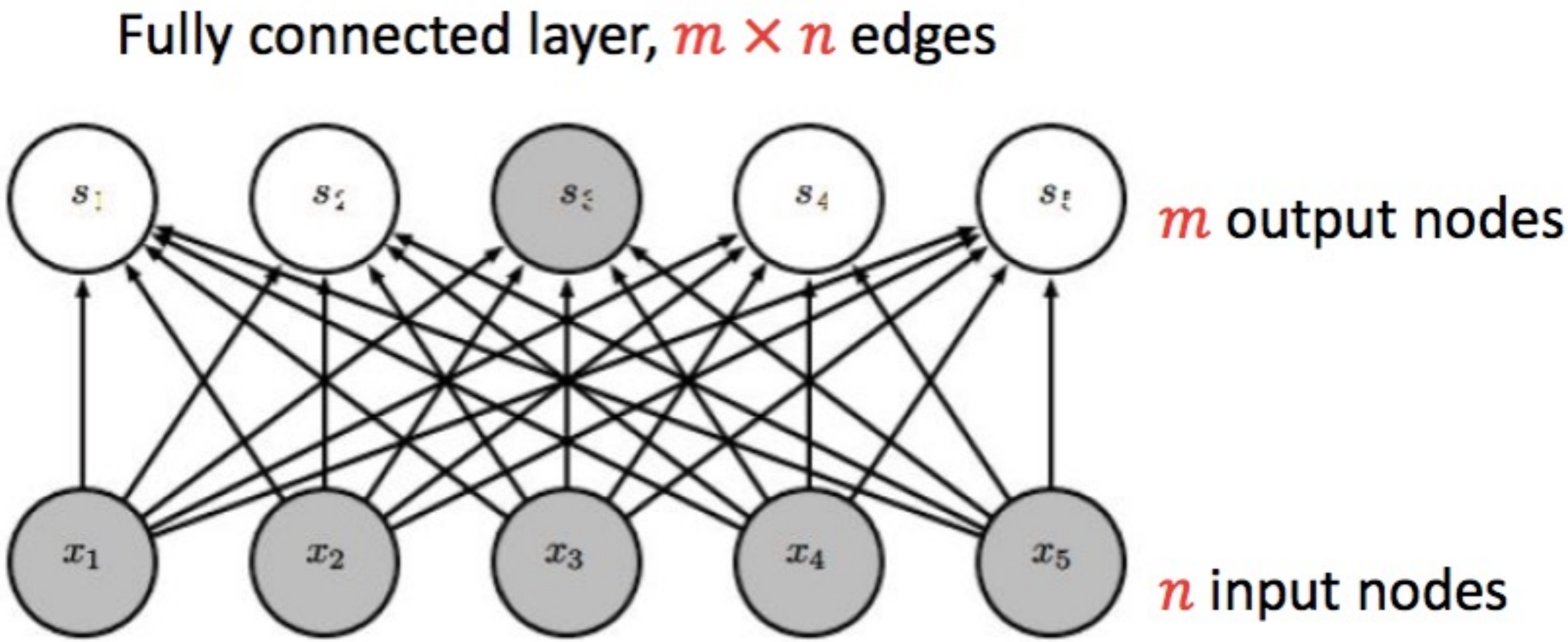
Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

Advantage of Convolutional Layer

Attendance Code: 482339

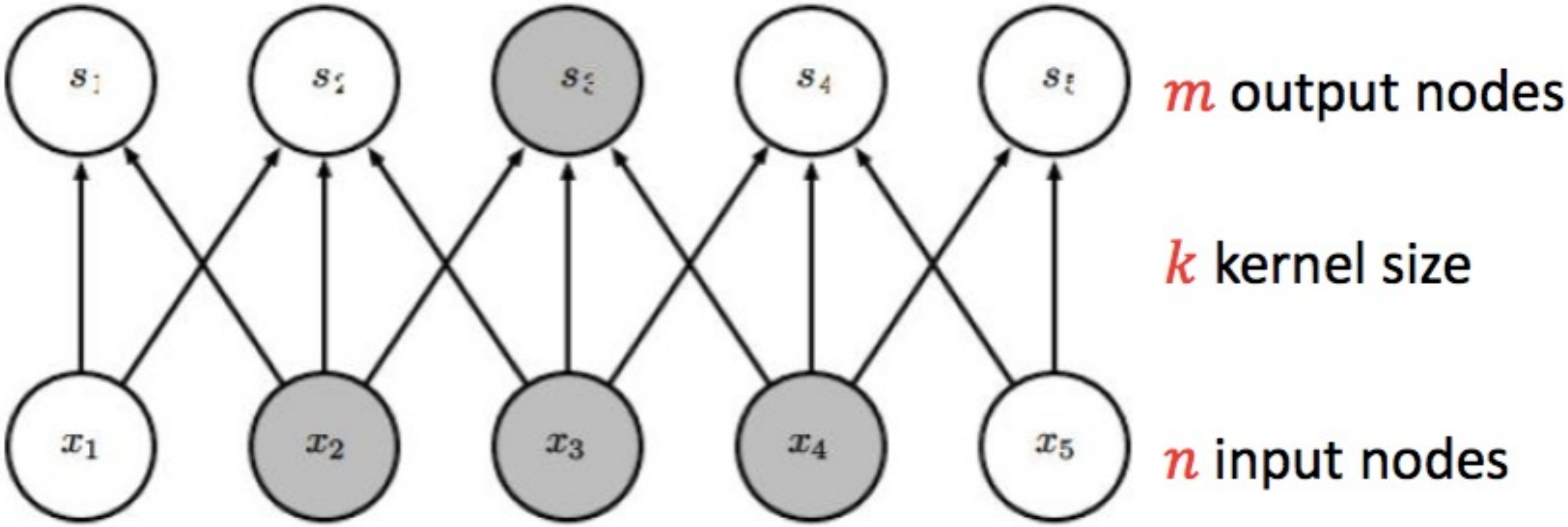
Advantage: sparse interaction



Attendance Code: 482339

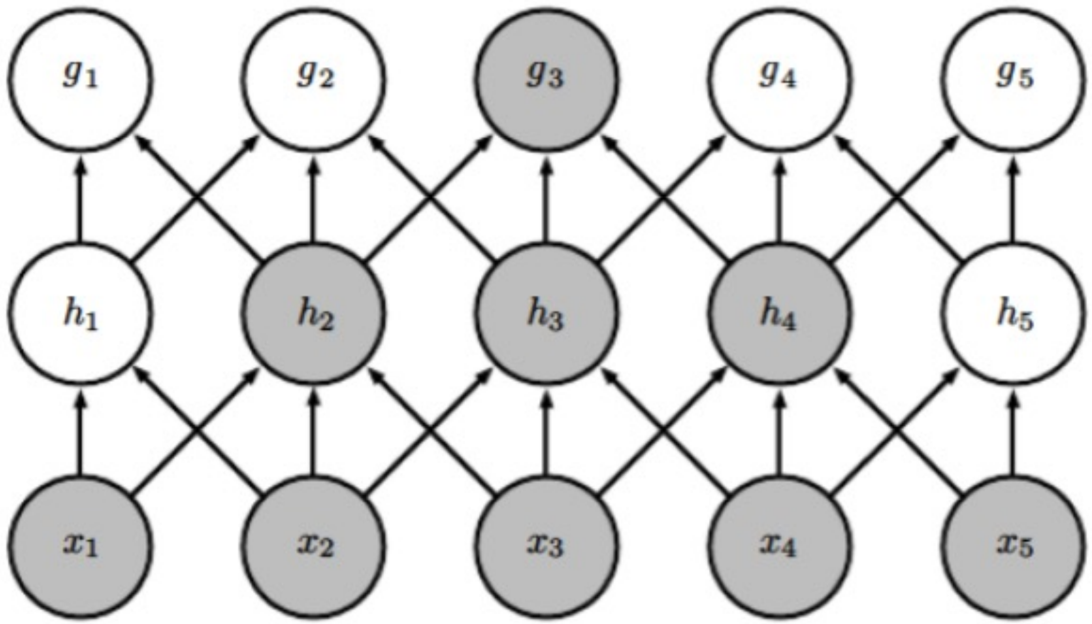
Advantage: sparse interaction

Convolutional layer, $\leq m \times k$ edges



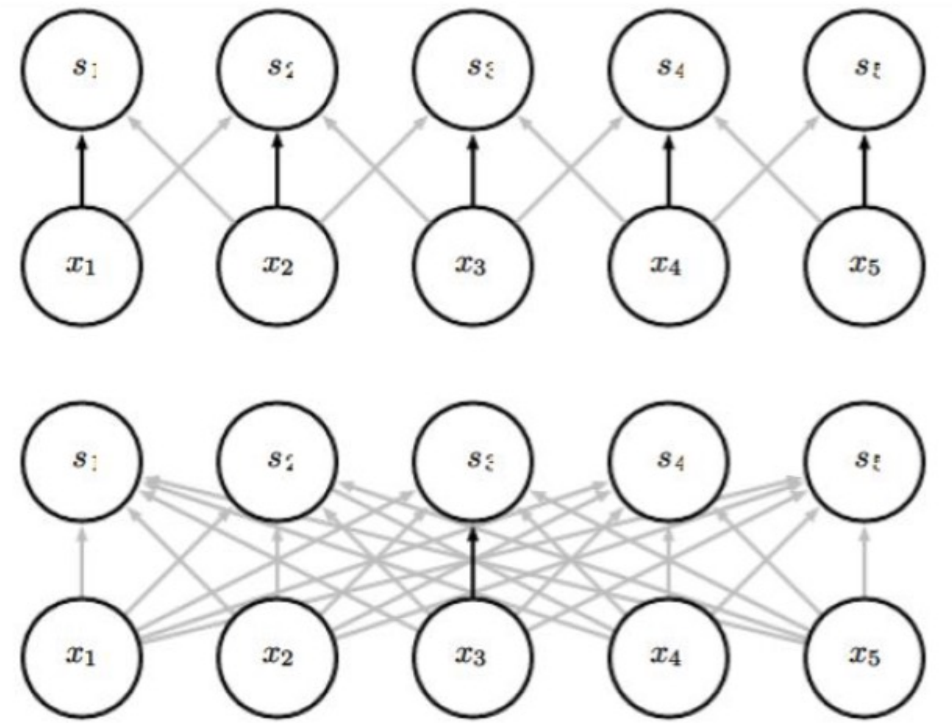
Advantage: sparse interaction

Multiple convolutional layers: larger receptive field



Attendance Code: 482339

Advantage: parameter sharing/weight tying



The same kernel are used repeatedly. E.g., the black edge is the same weight in the kernel.

Figure from *Deep Learning,* by Goodfellow, Bengio, and Courville

Advantage: equivariant representations

- Equivariant: transforming the input = transforming the output
- Example: input is an image, transformation is shifting
- Convolution(shift(input)) = shift(Convolution(input))
- Useful when care only about the existence of a pattern, rather than the location

Zero-Padding Layer

A zero-padding layer adds zeros around the borders of an input to preserve spatial dimensions after convolution and ensure that edge features are equally learned.

Attendance Code: 482339

Zero-Padding

filter

w	x
y	z

Input

a	b	c	d
e	f	g	h
i	j	k	l

0	**0**	**0**	**0**	**0**	**0**
0	a	b	c	d	**0**
0	e	f	g	h	**0**
0	i	j	k	l	**0**
0	**0**	**0**	**0**	**0**	**0**

Convolutional layer changes the input dimensions,
But sometimes we like the input and output of a convolutional layer to be of the same shape.

What's the shape of the resulting matrix?

ReLU Layer

A ReLU layer introduces nonlinearity by replacing negative values with zero, allowing the network to learn complex patterns and speeding up training through efficient gradient propagation.

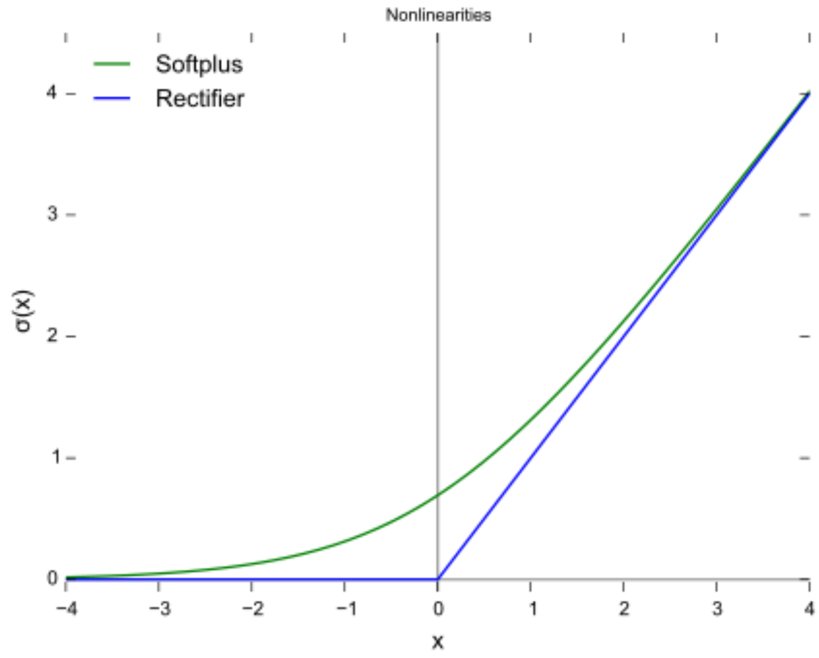
ReLU (**rectified linear unit**)

- **rectifier** is an activation function defined as the positive part of its argument

$$f(x) = \max(0,x)$$

- A smooth approximation to the rectifier is the analytic function

$$f(x) = \log(1+e^x)$$



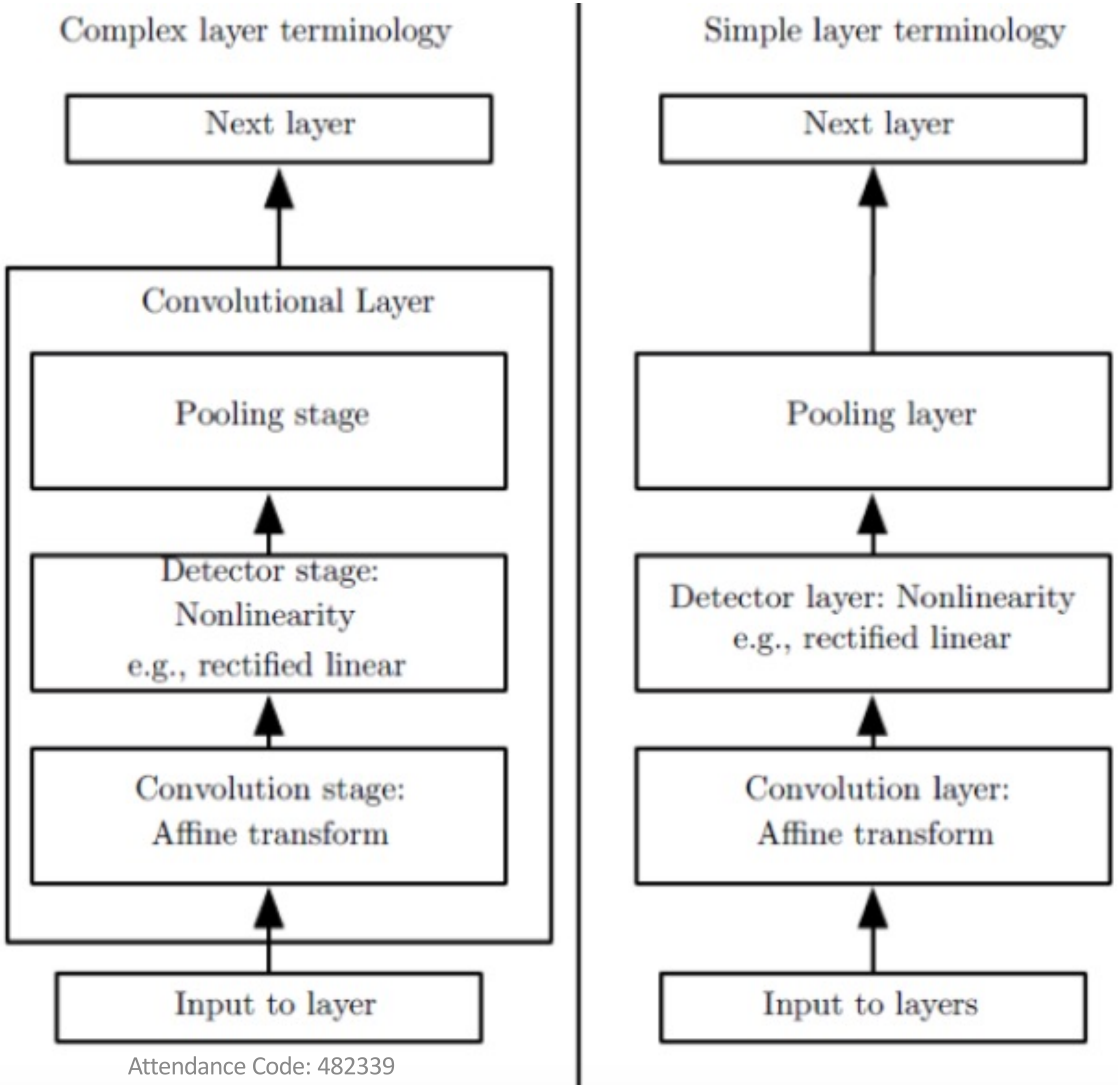
Pooling Layer

A pooling layer reduces the spatial dimensions of feature maps by summarizing local regions, which decreases computation, controls overfitting, and makes features more translation-invariant.

Pooling

- Pooling layer is frequently used in convolutional neural networks with the purpose to progressively reduce the spatial size of the representation to reduce the amount of features and the computational complexity of the network.

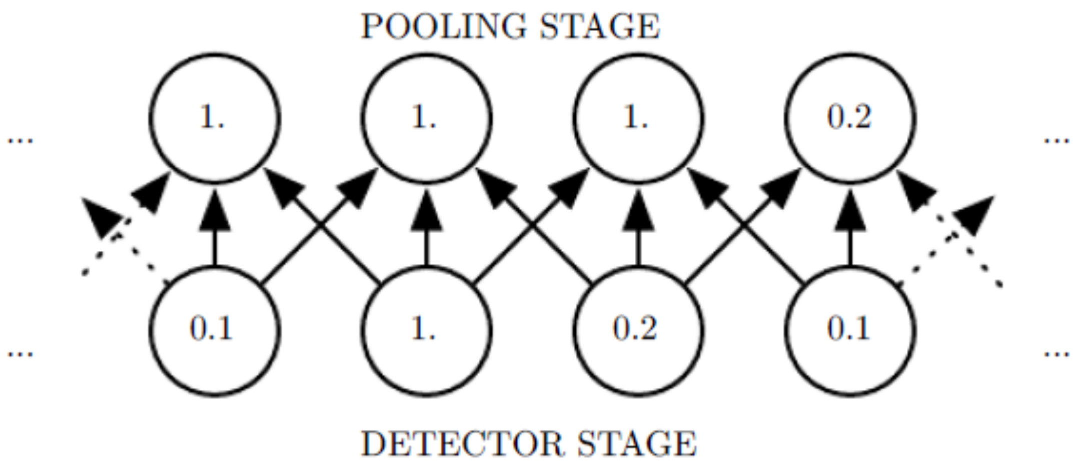
Terminology



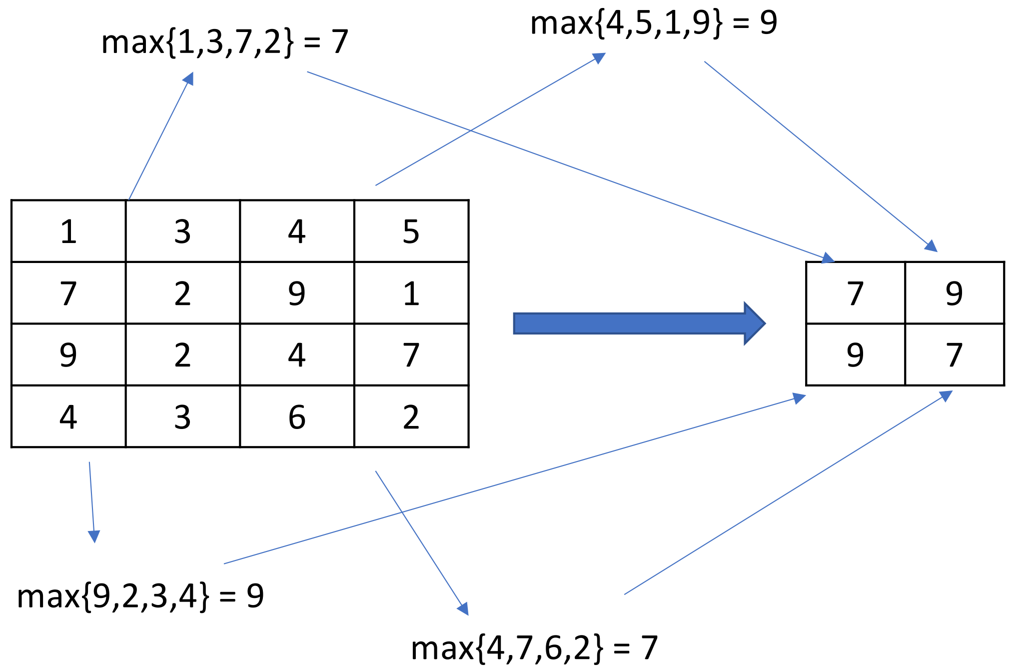
Attendance Code: 482339

Pooling

- Summarizing the input (i.e., output the max of the input)



Example: Max-pooling



2x2 max-pooling

Motivation from neuroscience

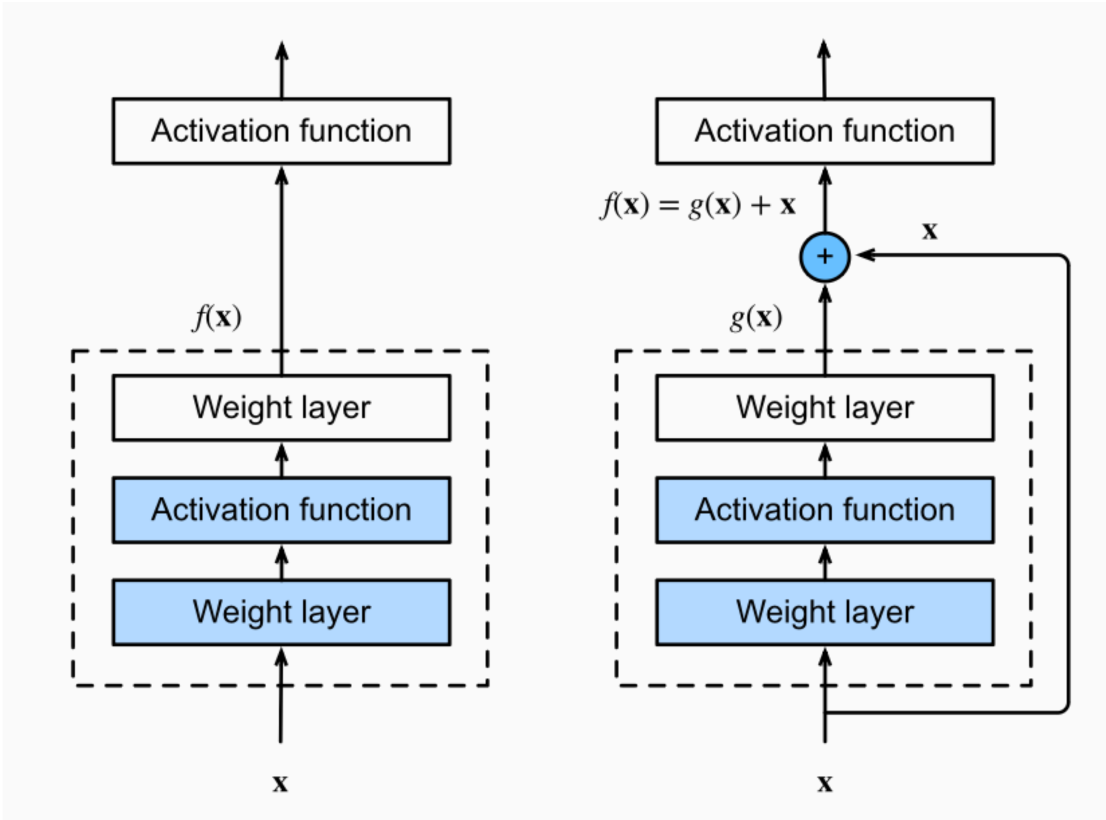
- David Hubel and Torsten Wiesel studied early visual system in human brain (V1 or primary visual cortex), and won Nobel prize for this

- V1 properties
  - 2D spatial arrangement
  - Simple cells: inspire convolution layers
  - Complex cells: inspire pooling layers

Residual Block

A Residual Block allows information to skip one or more layers through shortcut connections, helping deep networks train effectively by preventing vanishing gradients and preserving learned features.

Residual Block



regular block

residual block

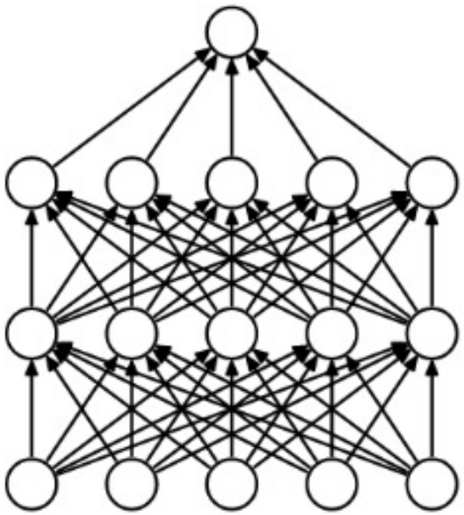
Attendance Code: 482339

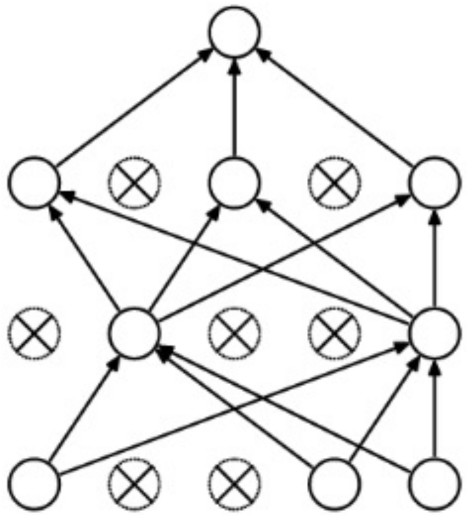
Dropout

A Dropout layer randomly disables a fraction of neurons during training to prevent overfitting and encourage the network to learn more robust, generalized features.

Attendance Code: 482339

Dropout


(a) Standard Neural Net

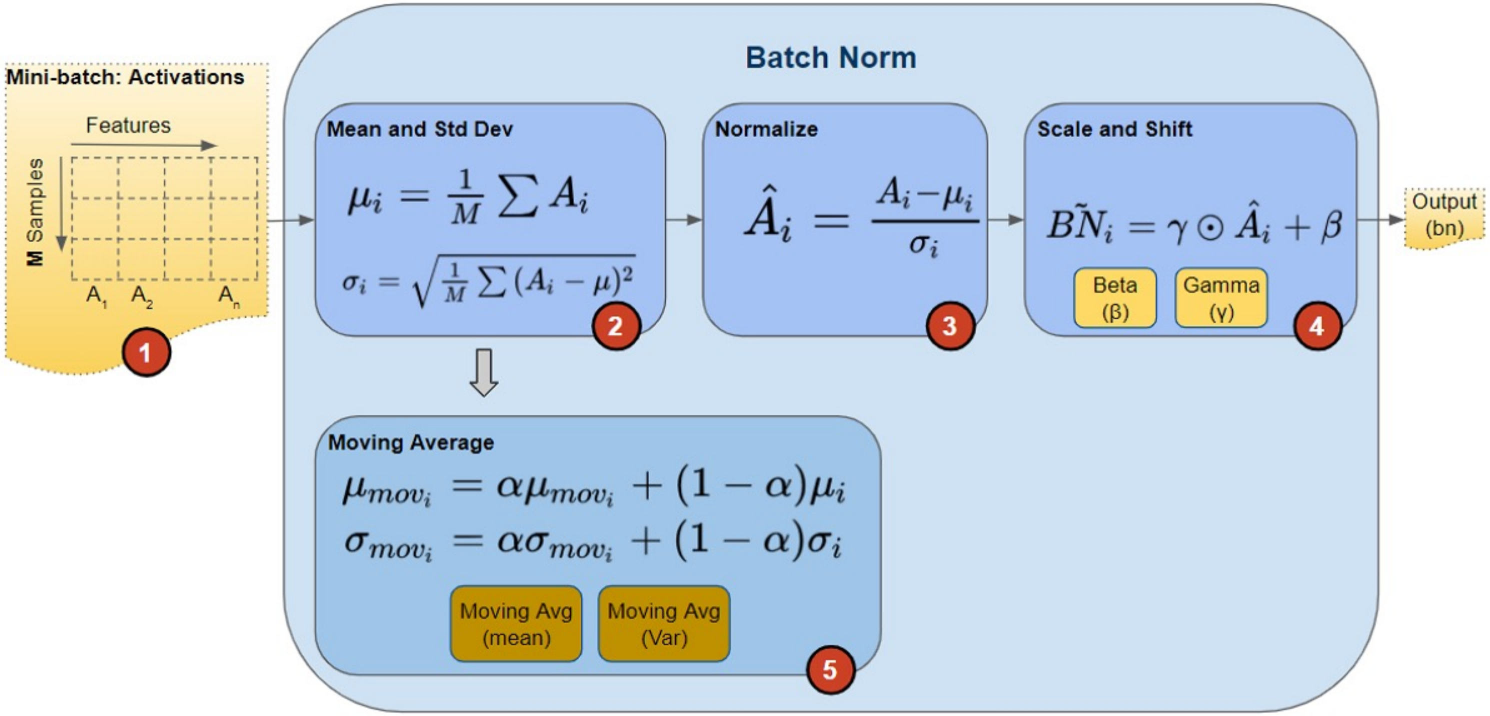

(b) After applying dropout.

Attendance Code: 482339

BatchNorm Layer

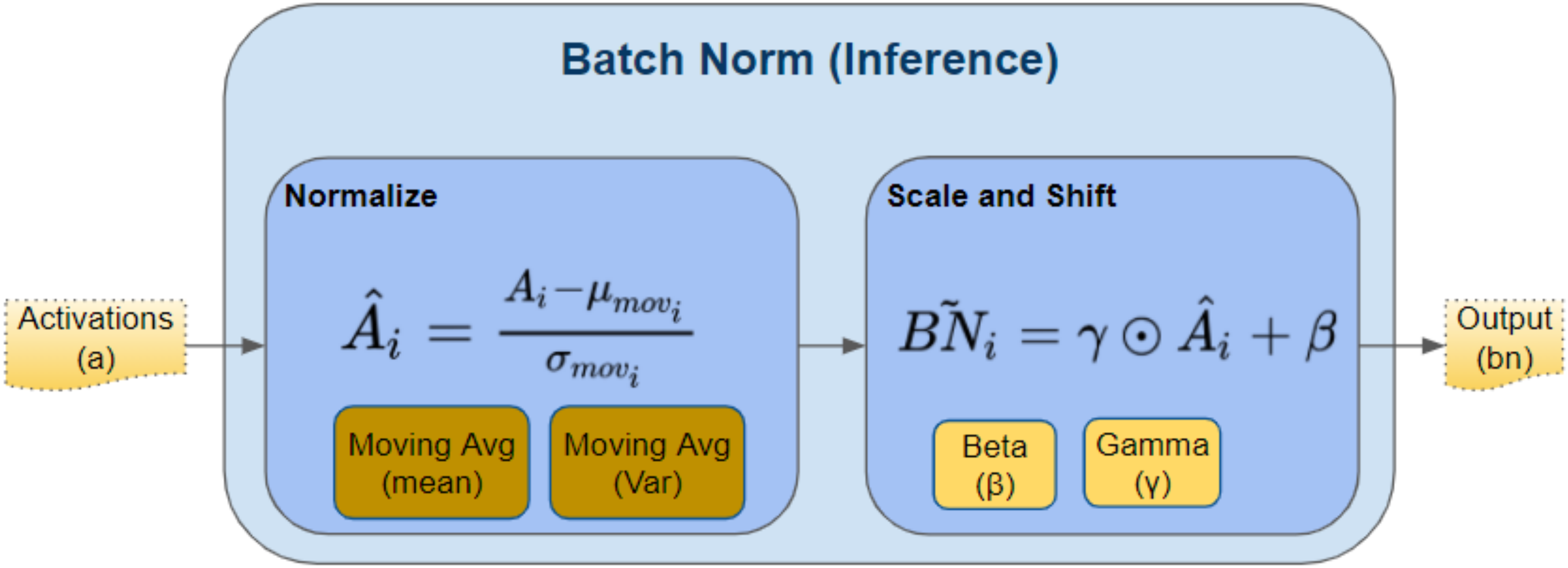
A Batch Normalization layer normalizes activations within a mini-batch to stabilize learning, speed up convergence, and reduce sensitivity to network initialization.

BatchNorm -- training



Attendance Code: 482339

BatchNorm -- inference



Attendance Code: 482339

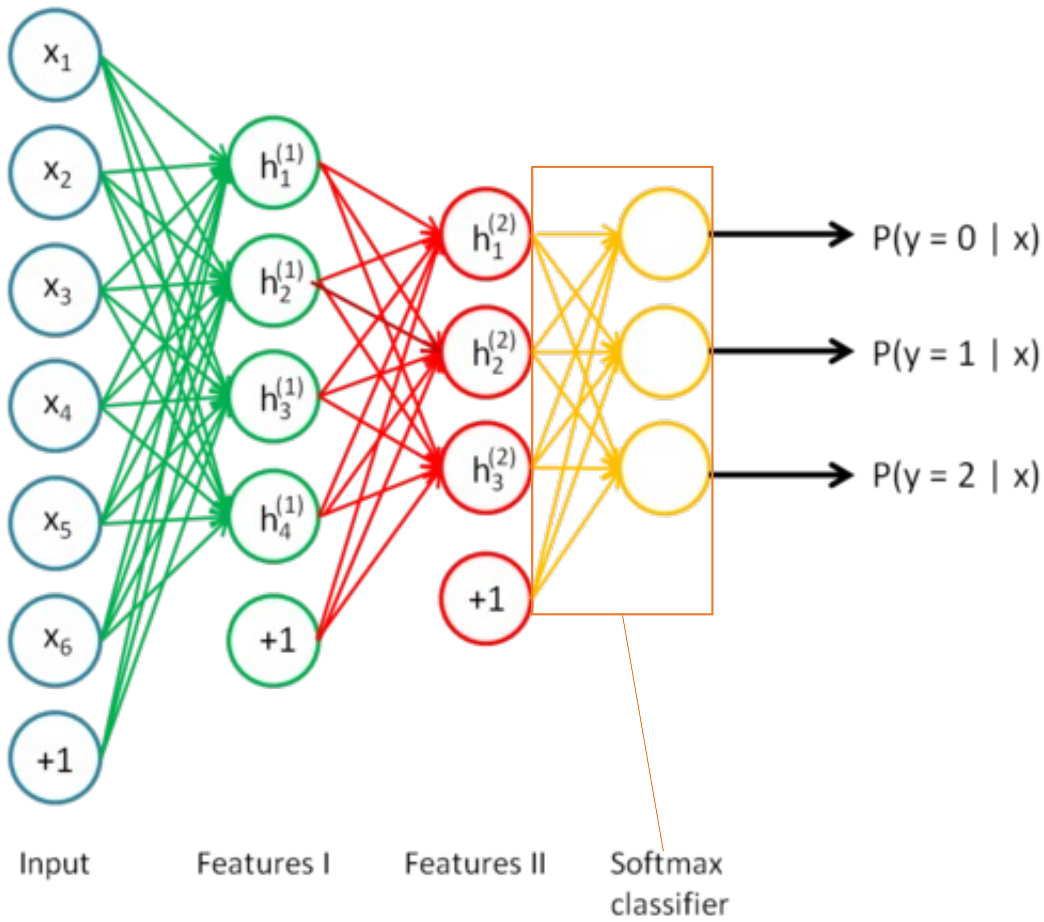
Softmax Layer

A Softmax layer converts raw output scores into probability distributions over classes, enabling multiclass classification by ensuring all outputs sum to one.

Softmax

- Recall that logistic regression produces a decimal between 0 and 1.0. For example, a logistic regression output of 0.8 from an email classifier suggests an 80% chance of an email being spam and a 20% chance of it being not spam. Clearly, the sum of the probabilities of an email being either spam or not spam is 1.0.
- **Softmax** extends this idea into a multi-class world. That is, Softmax assigns decimal probabilities to each class in a multi-class problem. Those decimal probabilities must add up to 1.0. This additional constraint helps training converge more quickly than it otherwise would.

Softmax

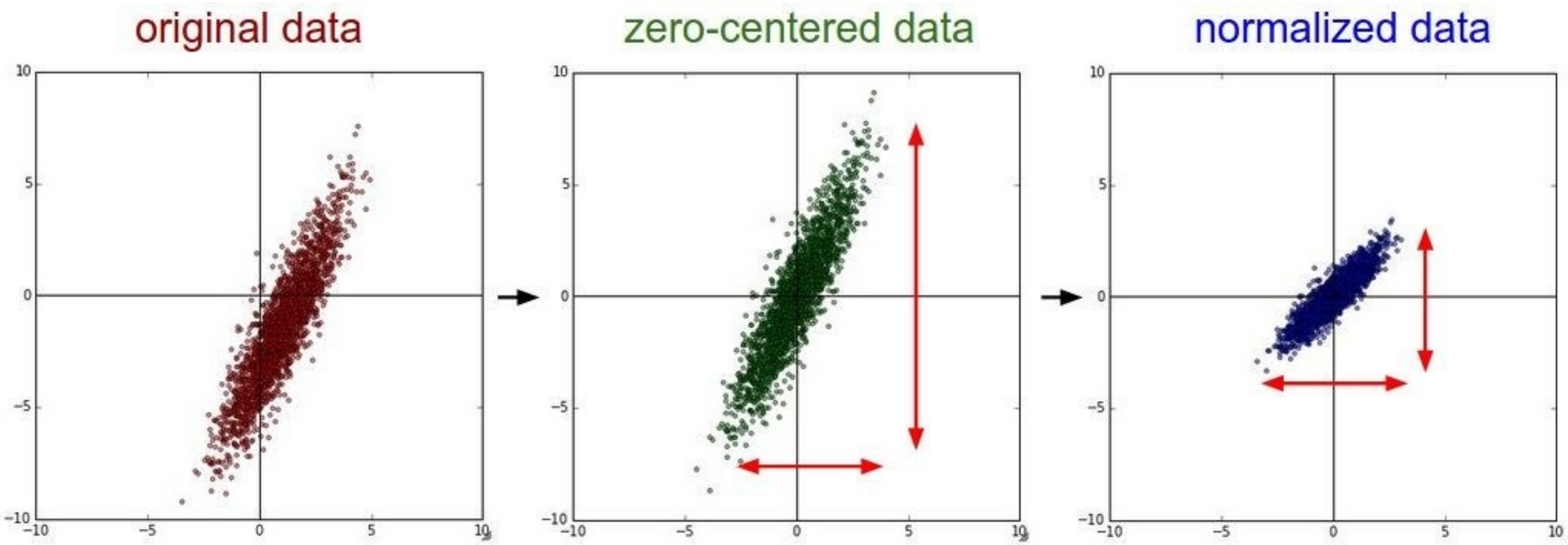

x_1
x_2
x_3
x_4
x_5
x_6
+1
$h_1^{(1)}$
$h_2^{(1)}$
$h_3^{(1)}$
$h_4^{(1)}$
+1
$h_1^{(2)}$
$h_2^{(2)}$
$h_3^{(2)}$
+1
$P(y = 0 \mid x)$
$P(y = 1 \mid x)$
$P(y = 2 \mid x)$
Input
Features I
Features II
Softmax classifier

Preprocessing data

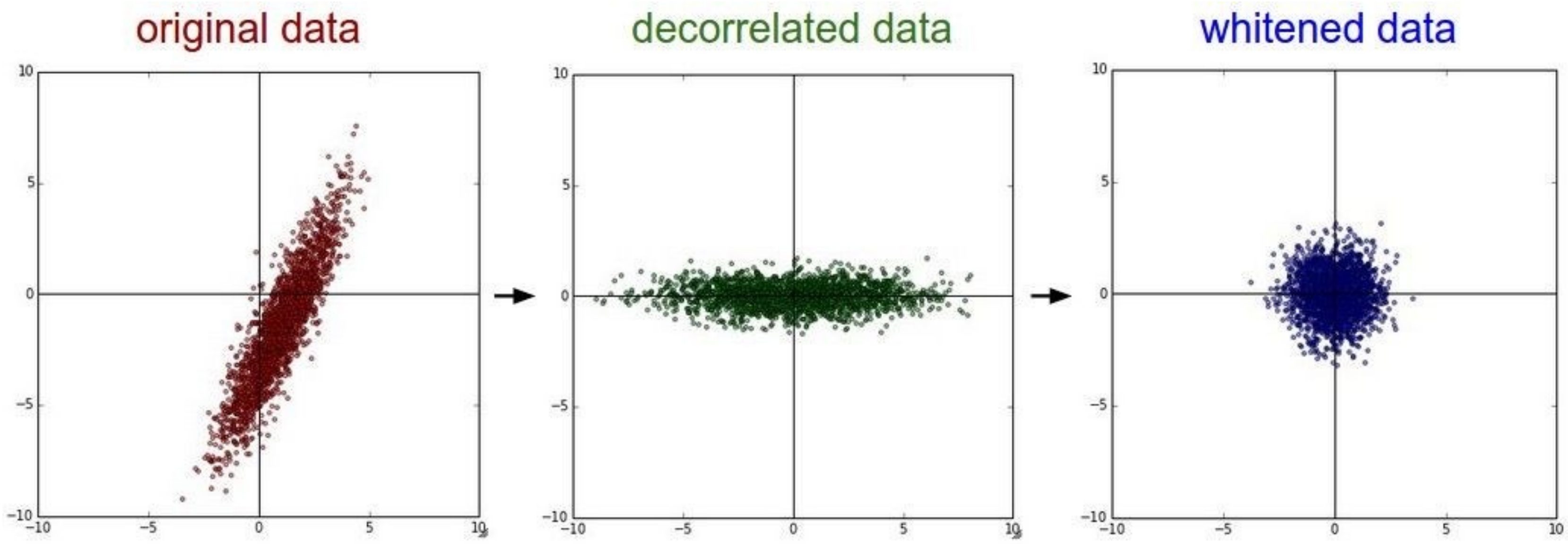
We need to preprocess data to clean, normalize, and structure raw inputs so that machine learning models can learn effectively, reduce noise, and converge faster with better accuracy.

Attendance Code: 482339

Preprocessing data



Preprocessing data



Attendance Code: 482339

Preprocessing data

e.g. consider CIFAR-10 example with [32,32,3] images

- Subtract the mean image (e.g. AlexNet) (mean image = [32,32,3] array)
- Subtract per-channel mean (e.g. VGGNet) (mean along each channel = 3 numbers)

Example: LeNet

Attendance Code: 482339

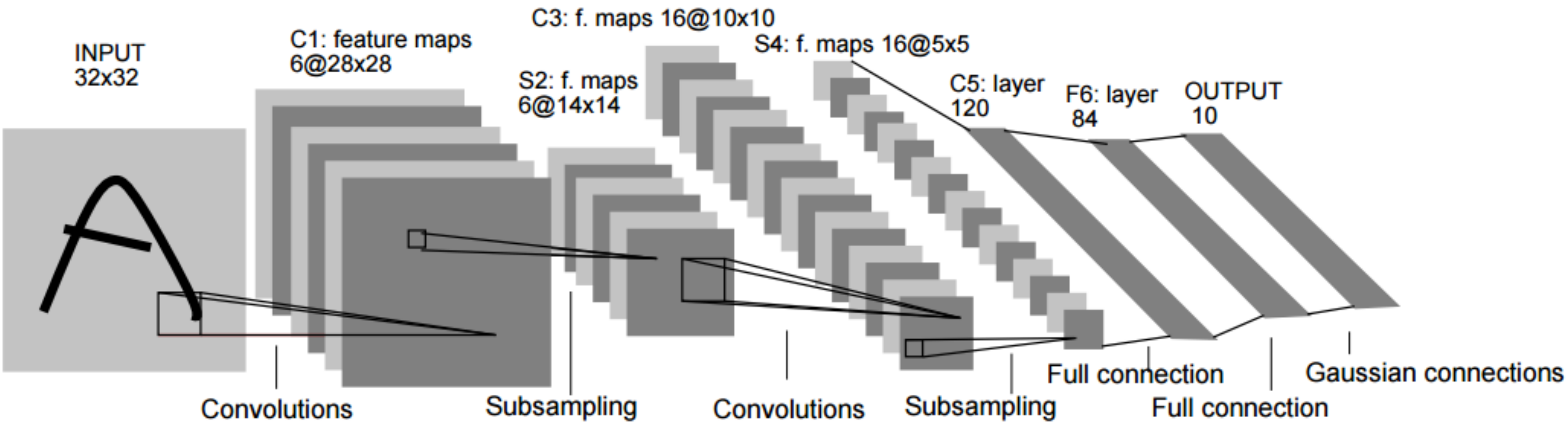
LeNet-5

- Proposed in *"Gradient-based learning applied to document recognition" , by Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, in Proceedings of the IEEE, 1998*

- Apply convolution on 2D images (MNIST) and use backpropagation

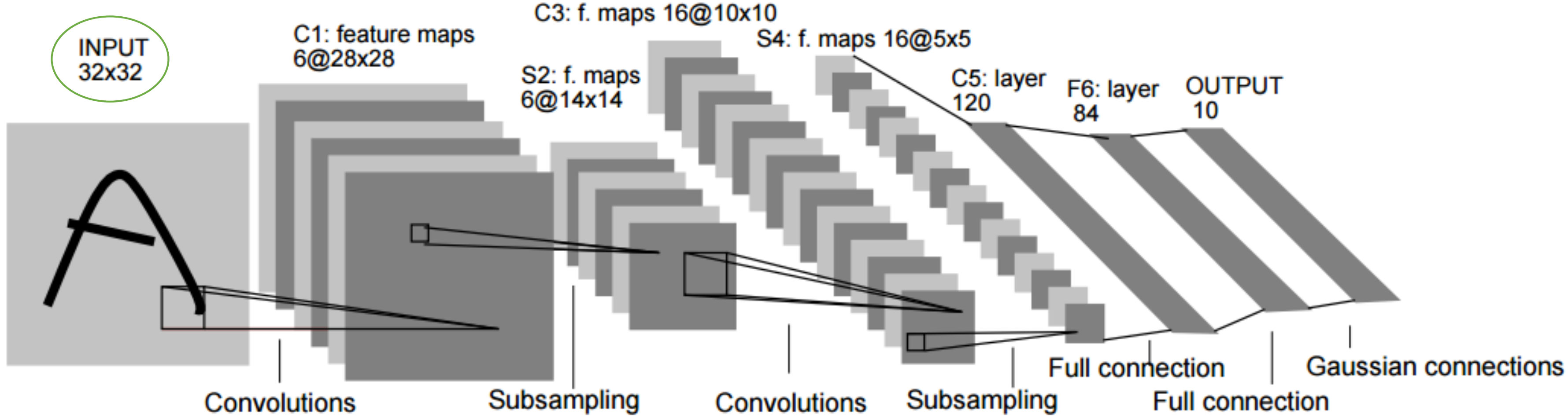
- Structure: 2 convolutional layers (with pooling) + 3 fully connected layers
    - Input size: 32x32x1
    - Convolution kernel size: 5x5
    - Pooling: 2x2

LeNet-5



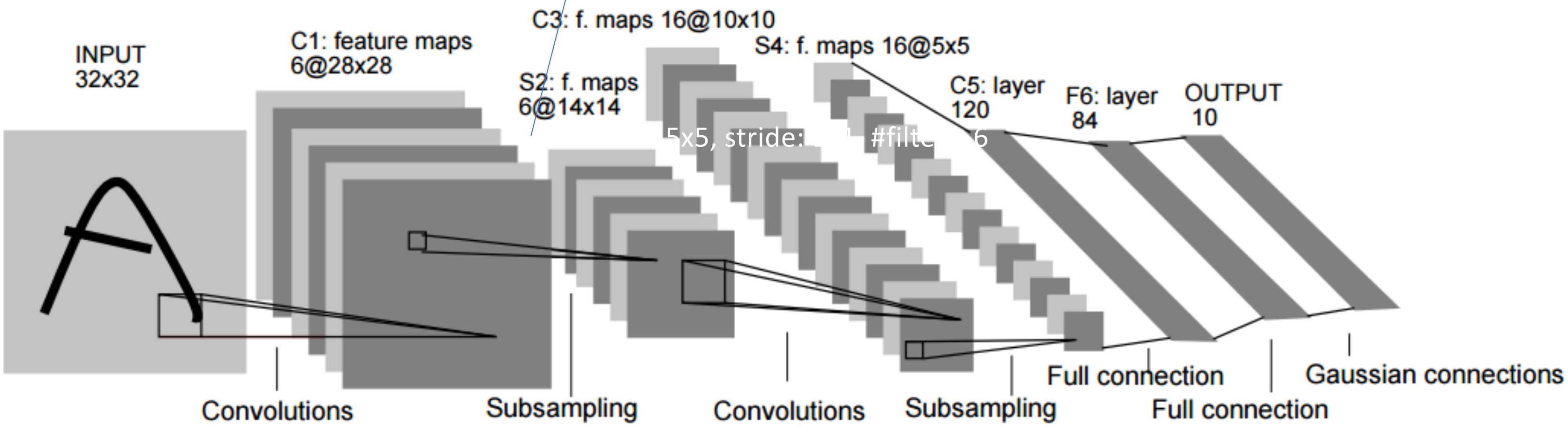
Attendance Code: 482339

LeNet-5


INPUT
32x32
C1: feature maps
6@28x28
S2: f. maps
6@14x14
C3: f. maps 16@10x10
S4: f. maps 16@5x5
C5: layer
120
F6: layer
84
OUTPUT
10
Convolutions
Subsampling
Convolutions
Subsampling
Full connection
Full connection
Gaussian connections

LeNet-5

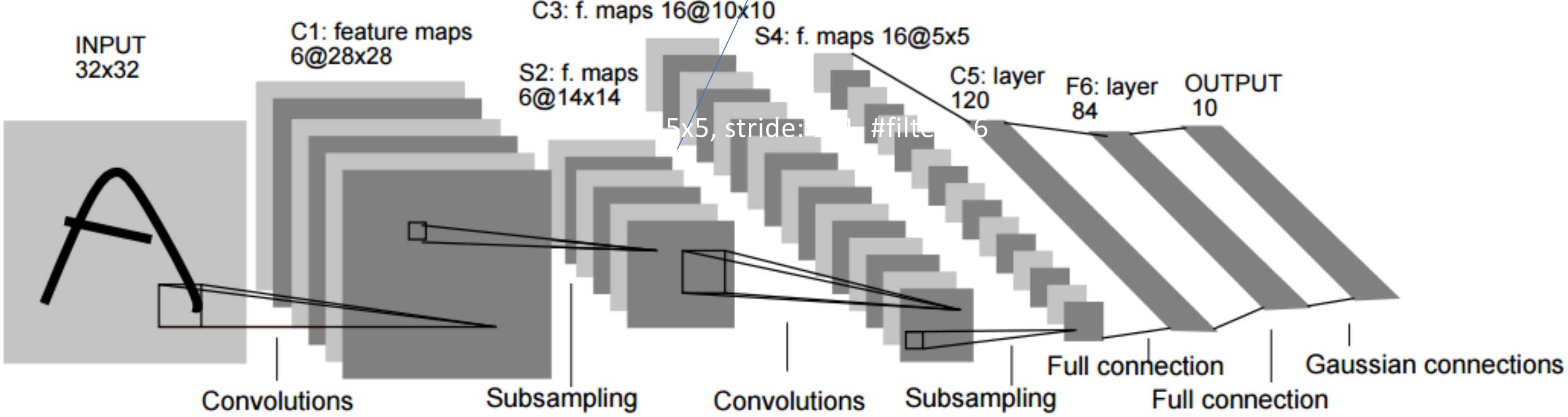
Pooling: 2x2, stride: 2



Attendance Code: 482339

LeNet-5

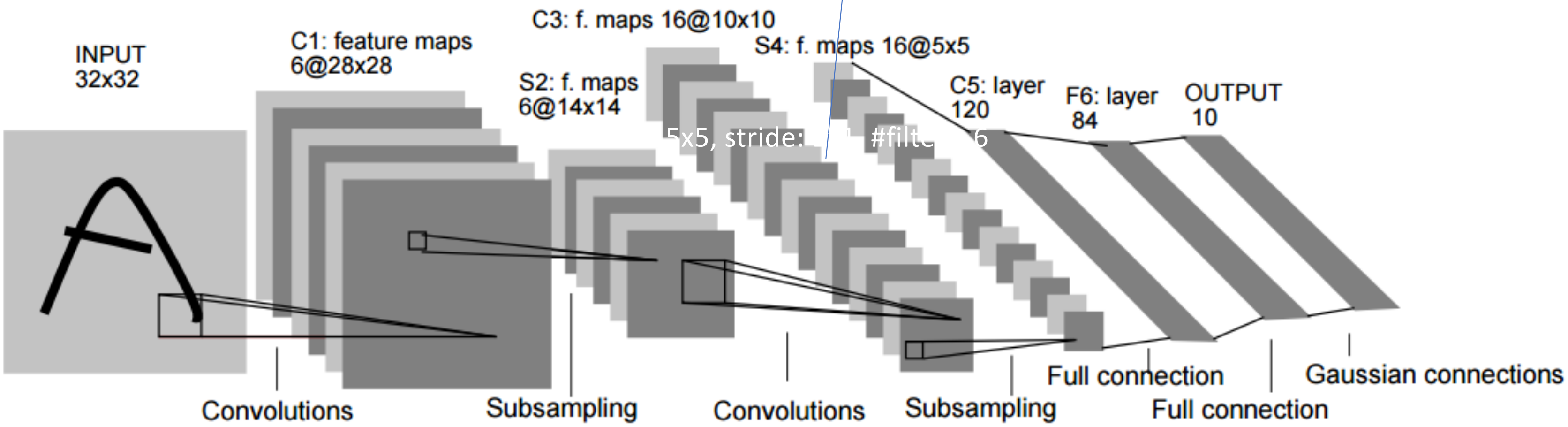
Filter: 5x5x6, stride: 1x1,
#filters: 16



Attendance Code: 482339

LeNet-5

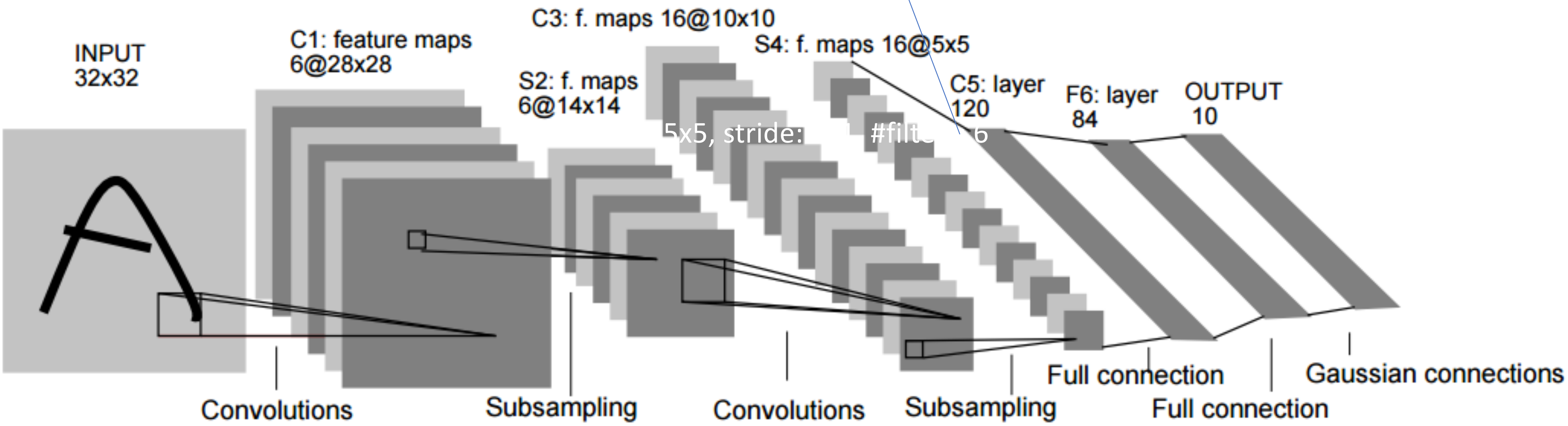
Pooling: 2x2, stride: 2



Attendance Code: 482339

LeNet-5

Weight matrix: 400x120



LeNet-5

Weight matrix: 120x84

Weight matrix: 84x10